

COSC264

Introduction to Computer Networks and the Internet

Reliable Data Transfer: ARQ Protocols

Dr Barry Wu

WRC

University of Canterbury

Barry.wu@canterbury.ac.nz

Learning Objectives

- At the end of the lectures, you will be able to:
 - Understand the need for reliable data transfer
 - Present an overview of the basic mechanisms of stop-and-wait ARQ, go-back-N ARQ, and selective repeat/reject* ARQ

* Selective reject is used in Data and Computer Communications by William Stallings
Selective repeat is used in Computer networking: a top-down approach featuring the Internet by James F. Kurose & Keith W. Ross and Computer Networks by Andrew S. Tanenbaum & David J. Wetherall

Outline

Reliable data transfer (RDT) protocols

- rdt 1.0: over a perfectly reliable channel
- rdt 2.0: over a channel with bit errors
- rdt 3.0: over a lossy channel with bit errors
- Pipelined reliable data transfer protocols

ARQ (Automatic Repeat reQuest)

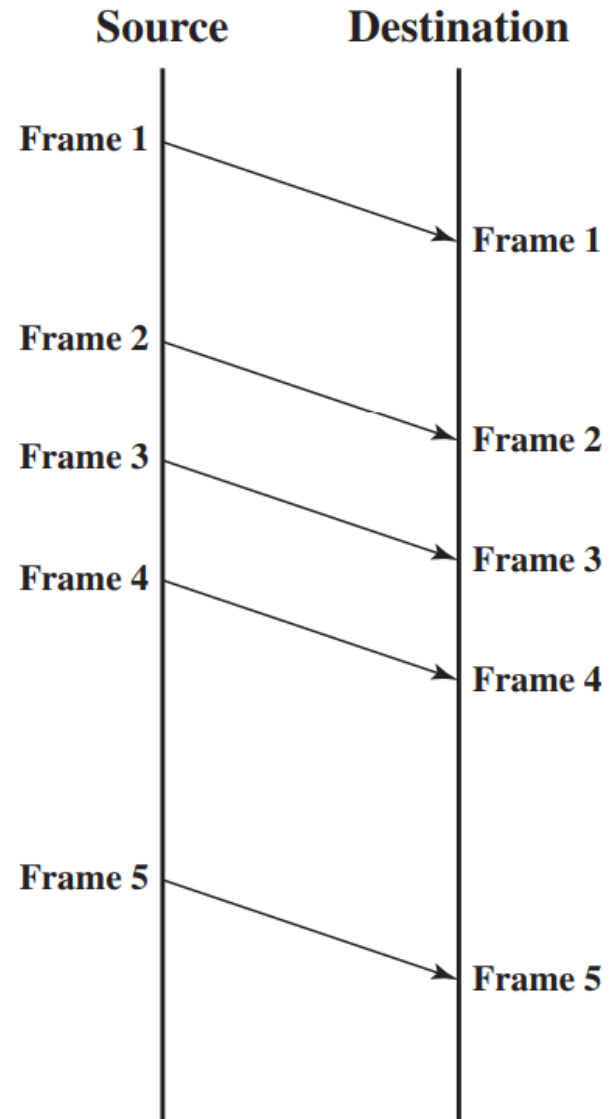
- Reliable data transfer protocols based on retransmission are known as ARQ (Automatic Repeat reQuest) protocols.
 - Stop-and-wait ARQ
 - Go-back-N ARQ
 - Selective repeat/reject ARQ

Reliable data transfer over a perfectly reliable channel

Error-Free Transmission: rdt 1.0

- A perfectly reliable channel
 - no bit errors
 - no out-of-order frames
 - no frame loss

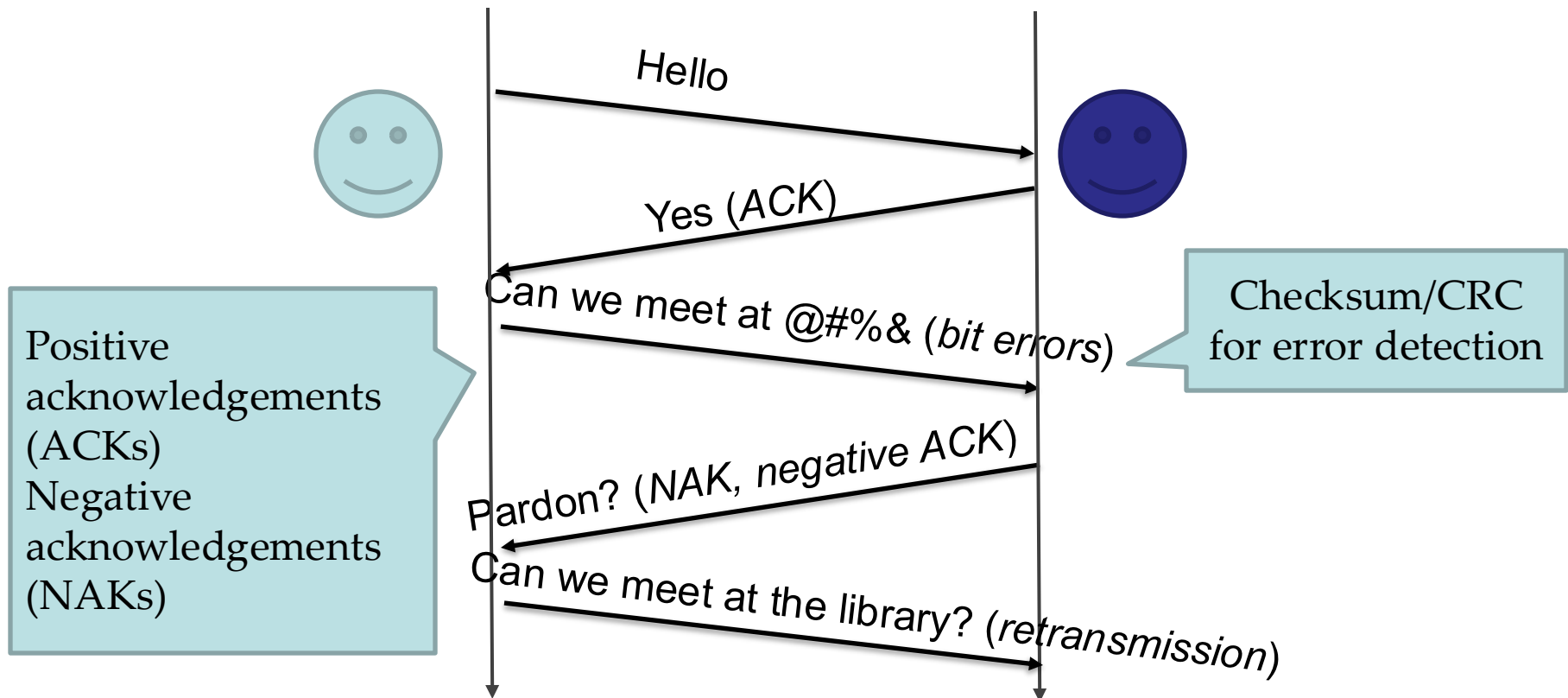
The sender sends data whenever data is available; the receiver receives data and delivers data to its upper-layer protocol.



Source: William Stallings, Data and Computer Communications, 10th edition, 2013, Figure 7.1.

Reliable data transfer over a lossless channel with bit errors

A channel with only bit errors



Stop-and-Wait Protocols: rdt 2.0

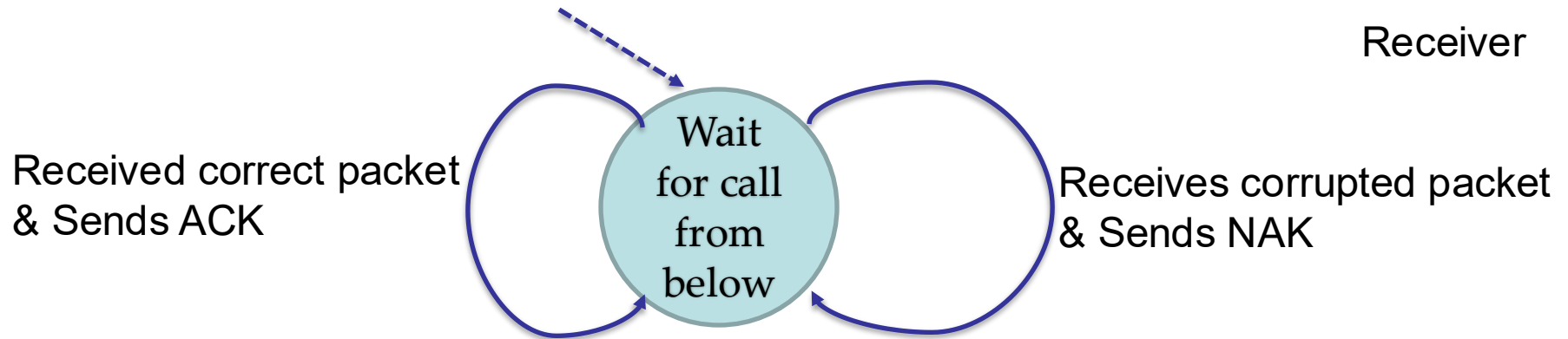
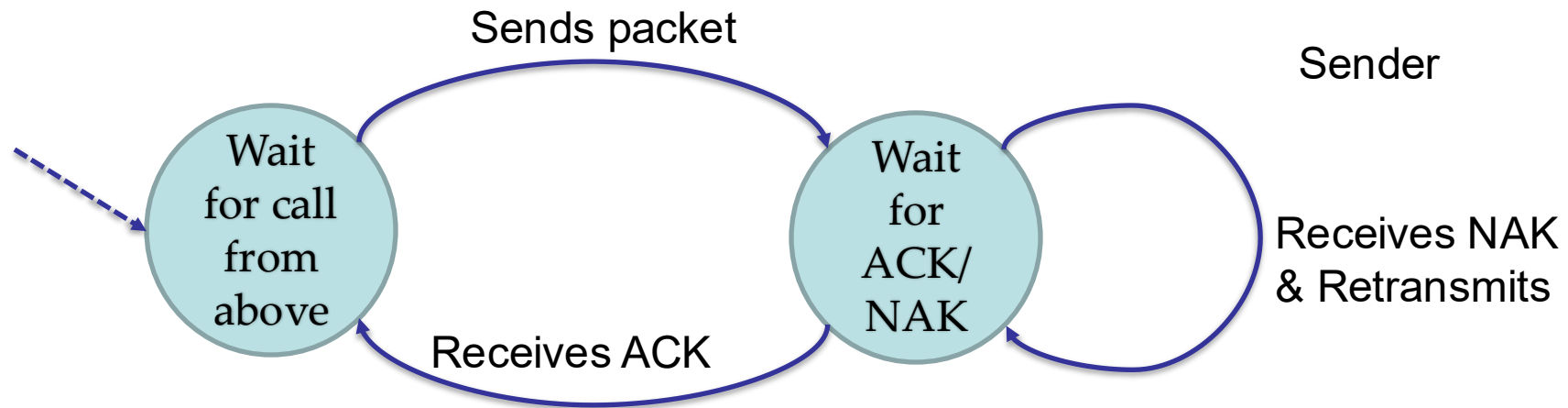
■ Sender

- When the packet is ready, sends it and waits for ACK/NAK
- If NAK is received, retransmits the last packet and waits for ACK/NAK
- If ACK is received, waits for the next packet to be ready

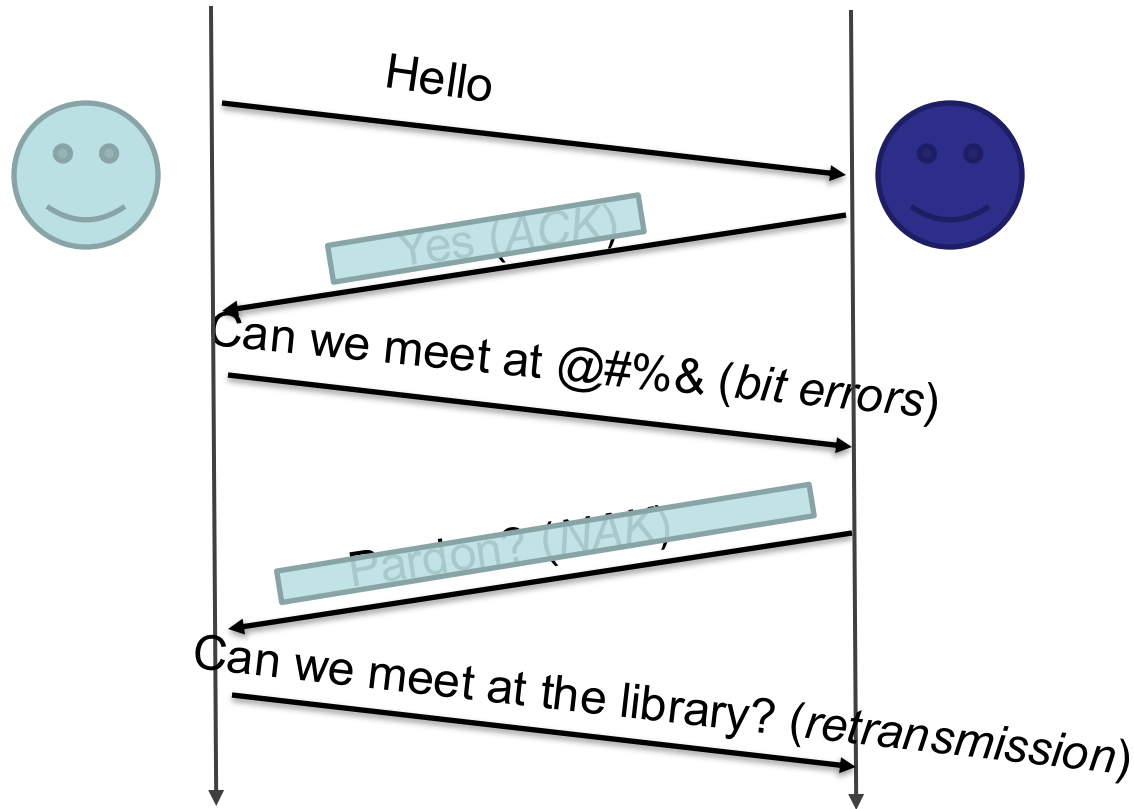
■ Receiver

- If the packet is received without error, sends ACK
- If the packet is received with error, sends NAK

rdt 2.0: FSM (Finite-state machine)

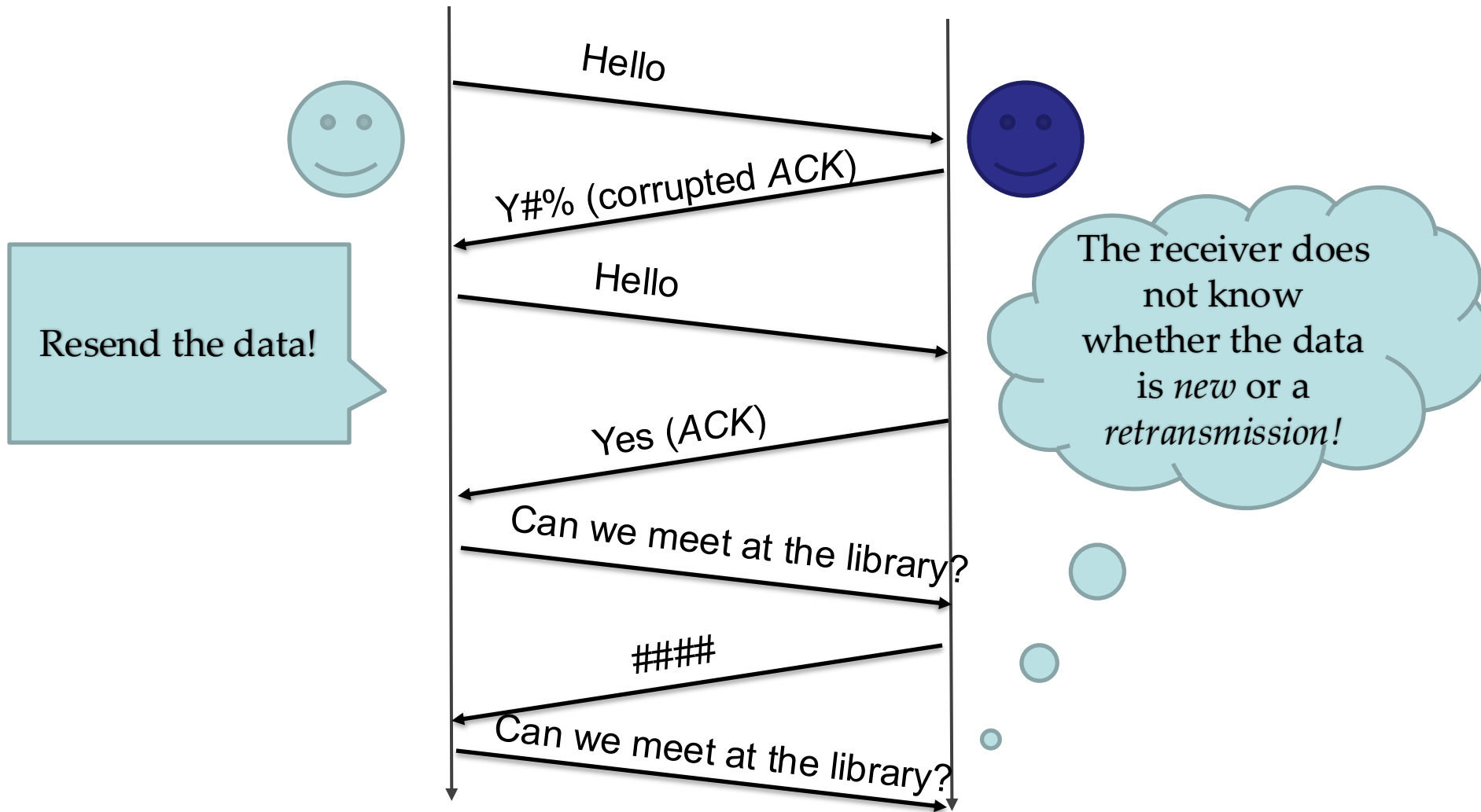


rdt 2.0: A Fatal Flaw

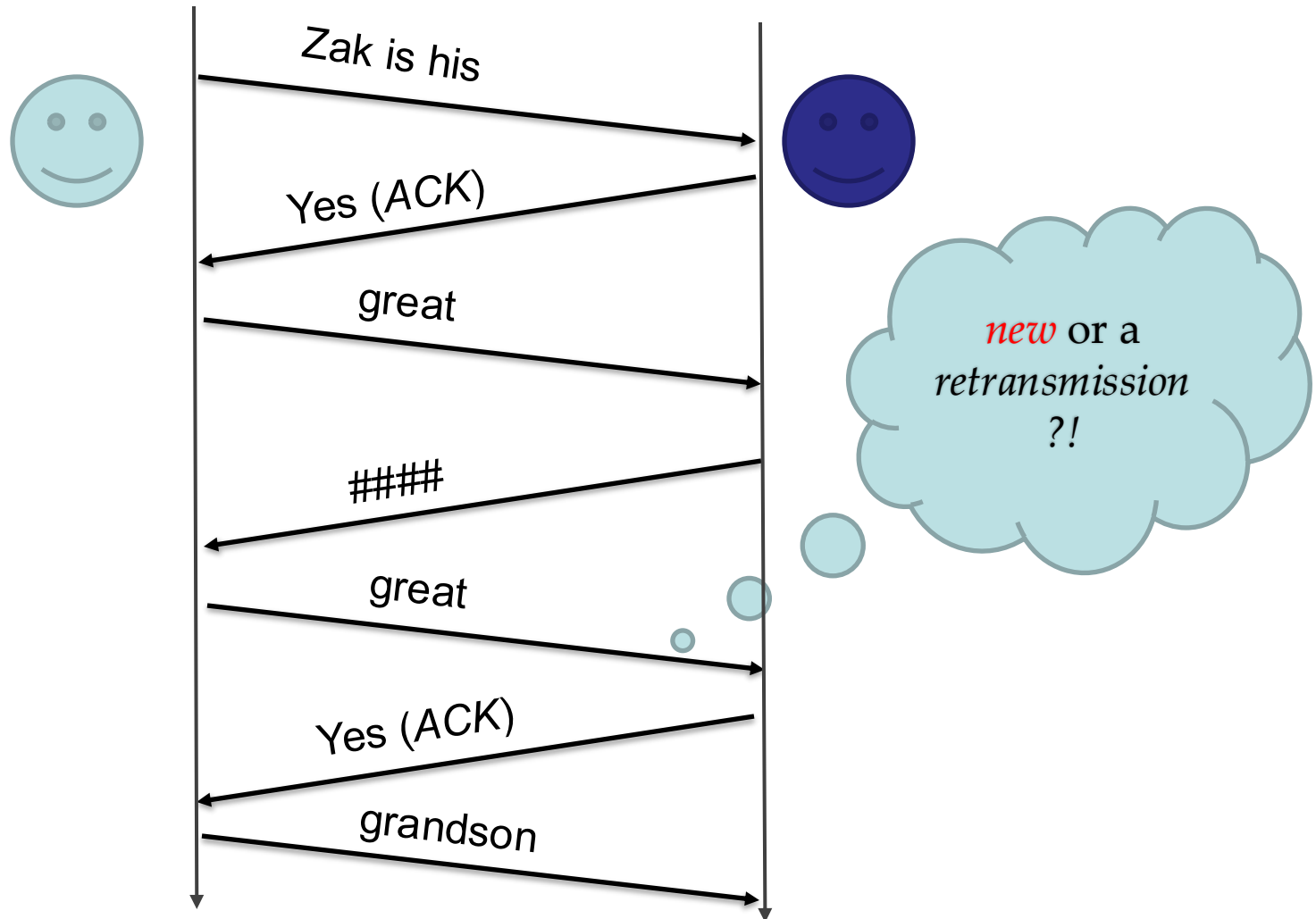


ACK/NAK can be corrupted!

rdt 2.0: Corrupted ACK/NAK



rdt 2.0: Corrupted ACK/NAK (cont.)

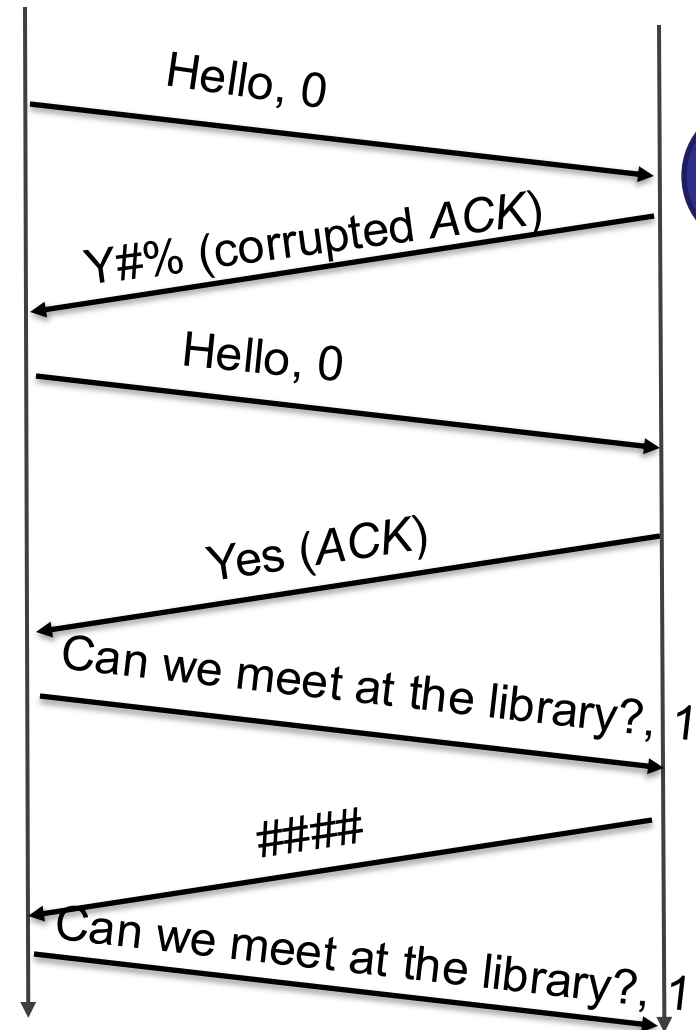


How to handle corrupted ACK/NAK: rdt 2.1

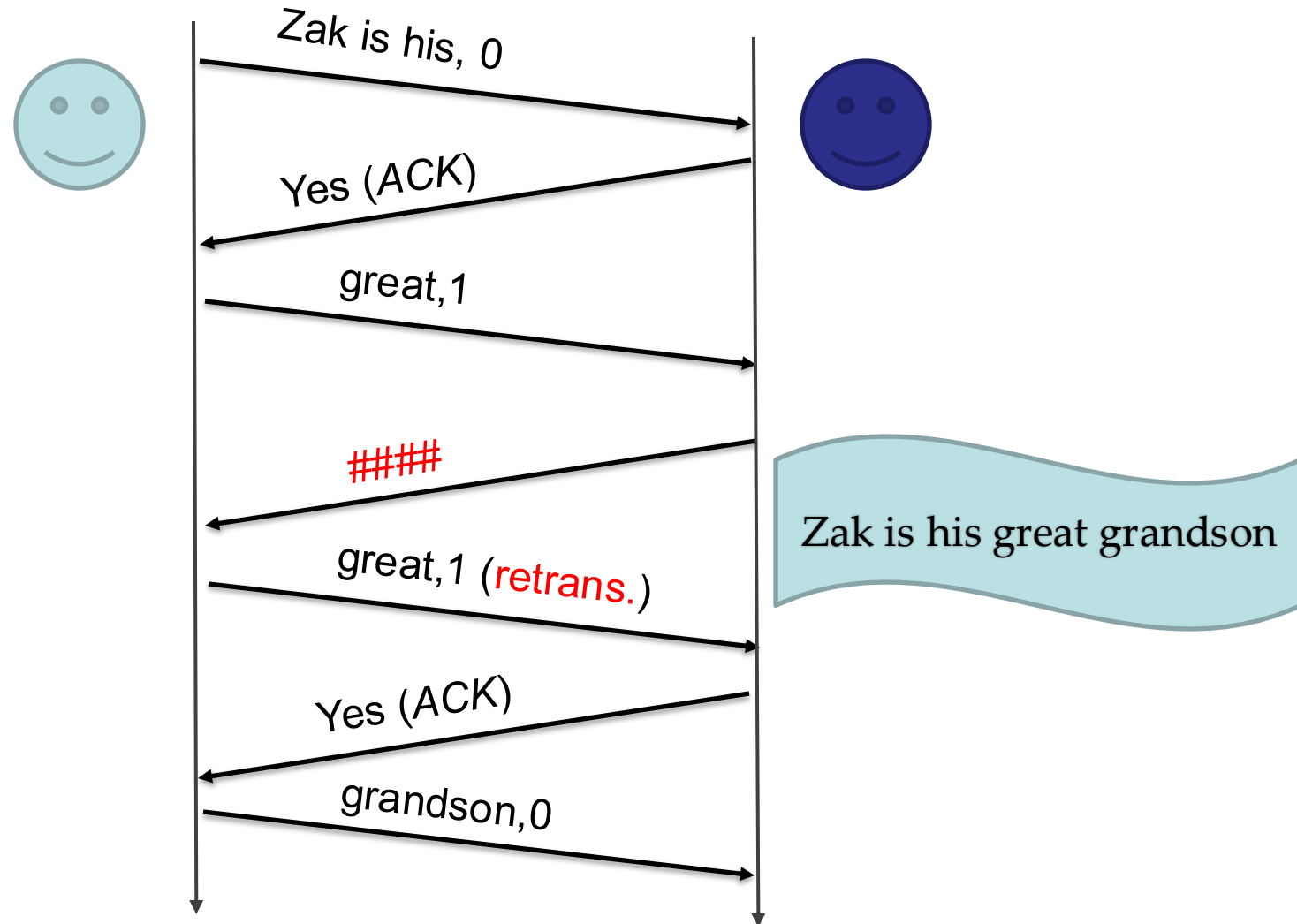
- For this simple case of a stop-and-wait protocol, add a new field (**1-bit sequence number**)

- retransmission (same as previous)
- new (different from previous)

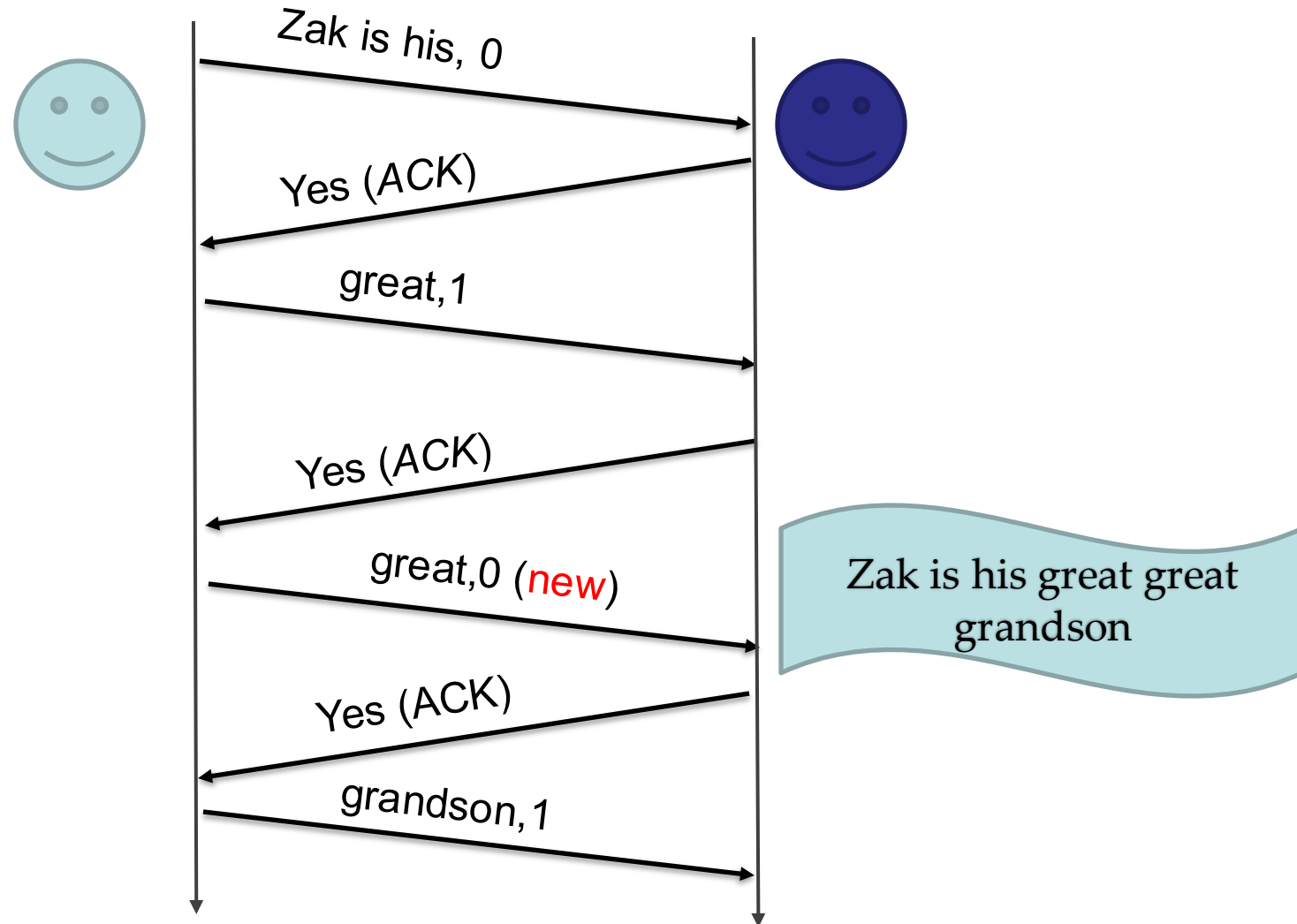
The receiver will not be confused!



How to handle corrupted ACK/NAK: rdt 2.1 (cont.)

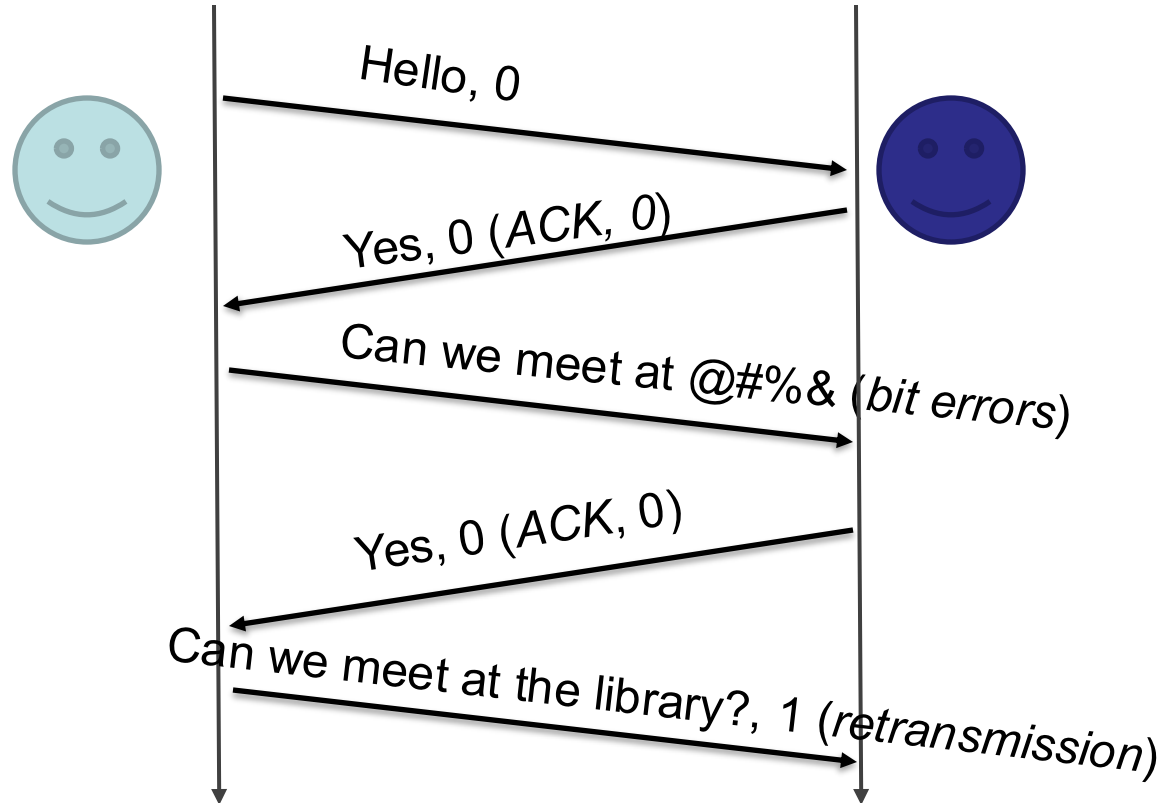


How to handle corrupted ACK/NAK: rdt 2.1 (cont.)

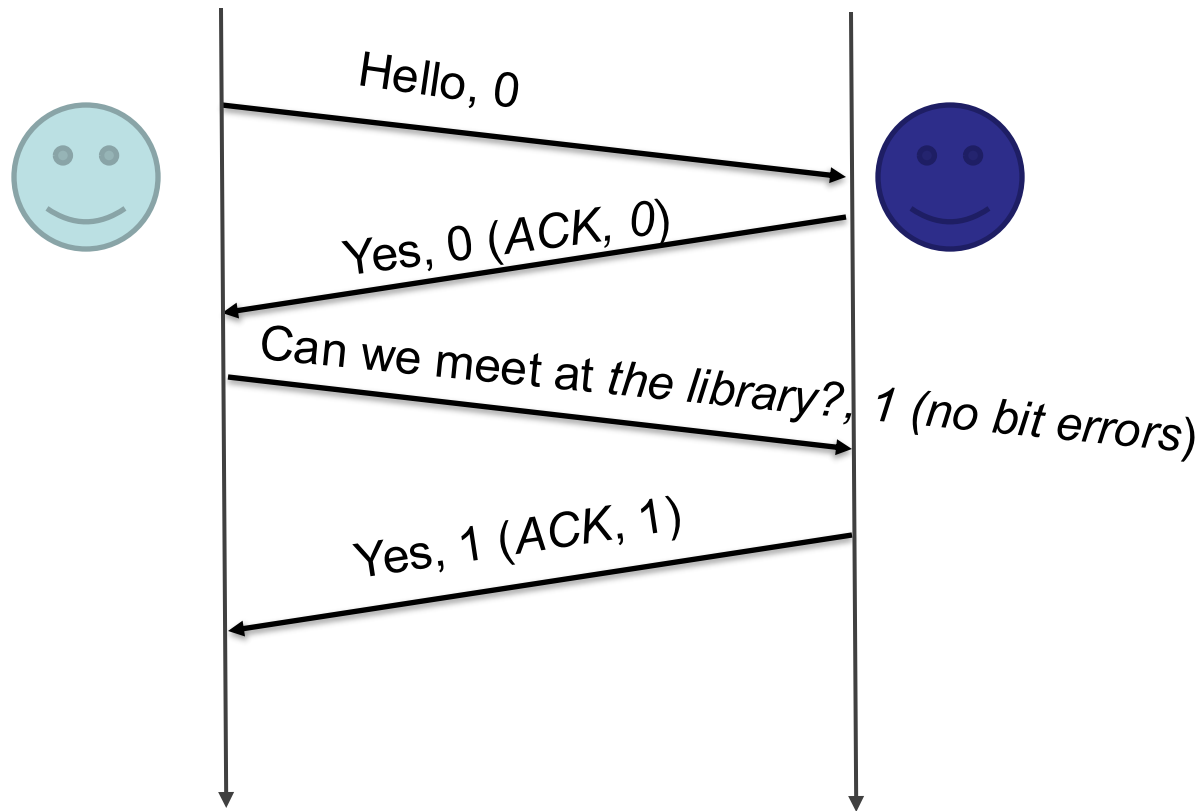


How to handle corrupted ACK/NAK: rdt 2.2

- NAK-free data transfer: use duplicate ACKs with the sequence number to replace NAK



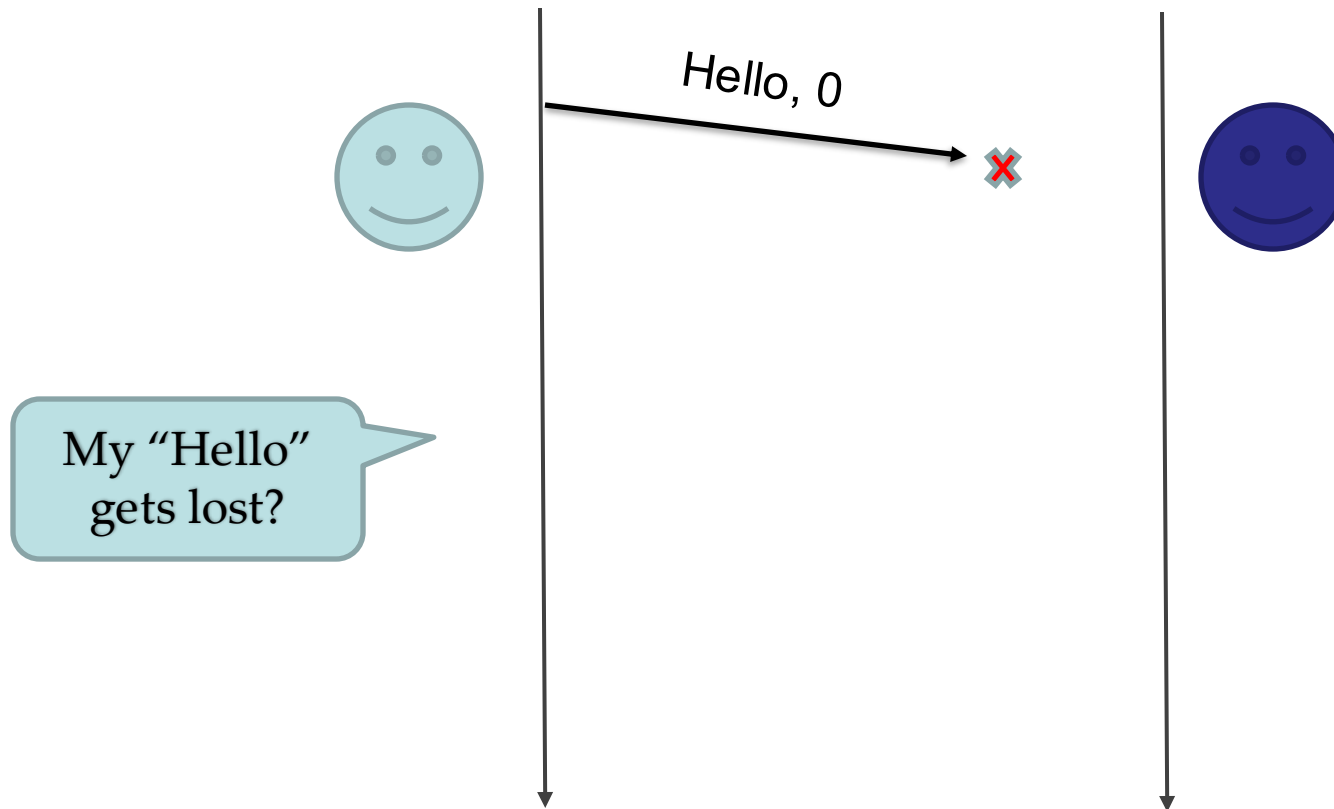
How to handle corrupted ACK/NAK: rdt 2.2 (cont.)



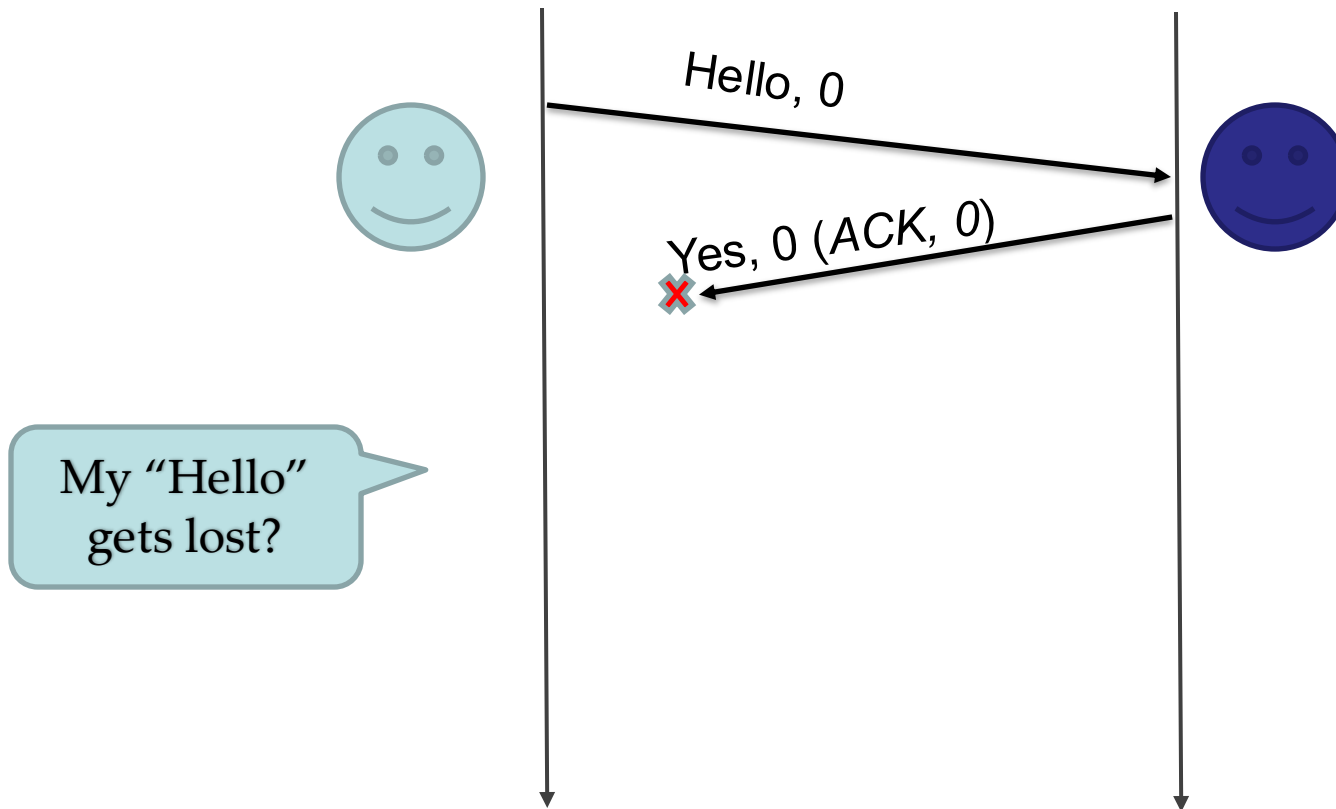
The receiver uses ACKs with the sequence number.

Reliable data transfer over a lossy channel with bit errors

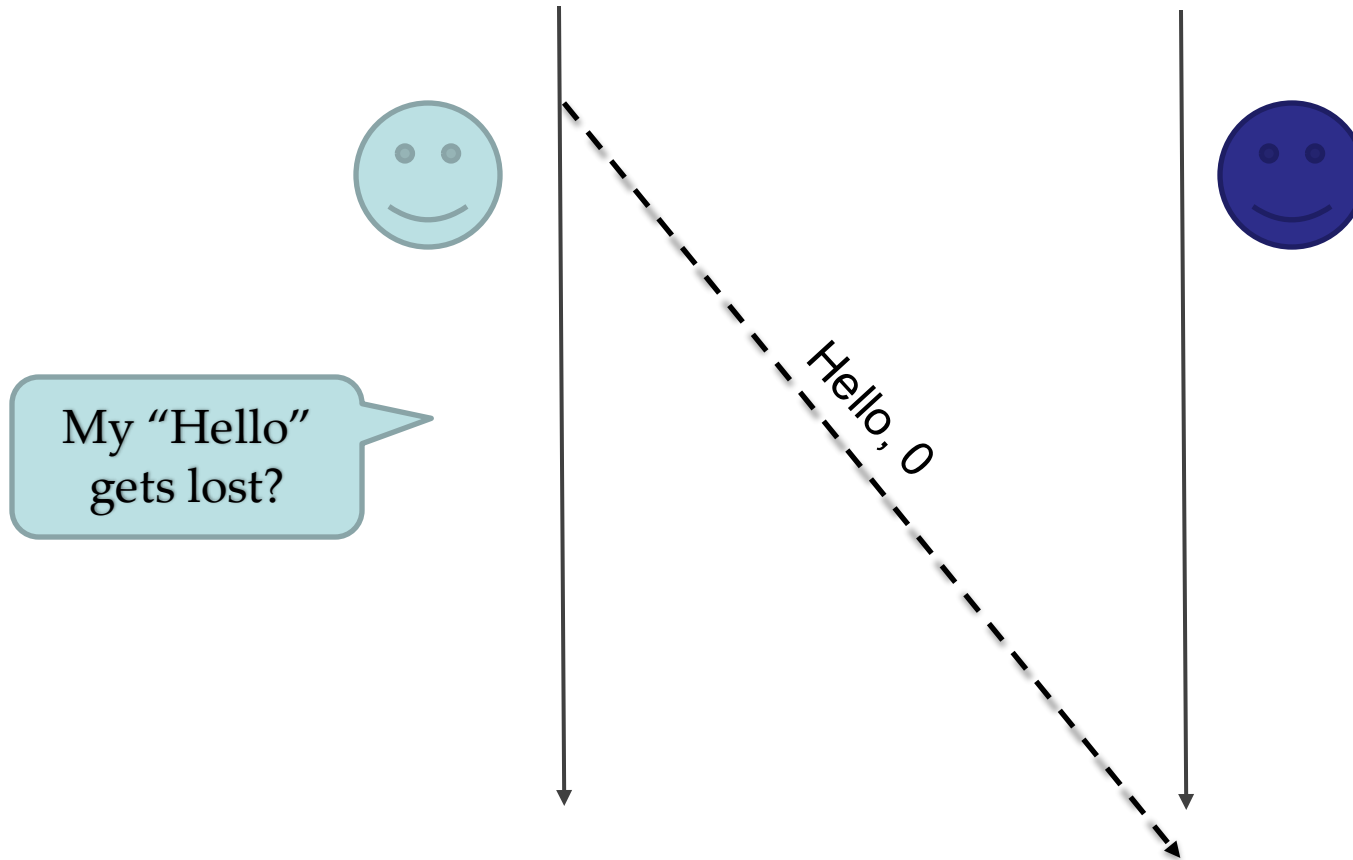
Data Loss



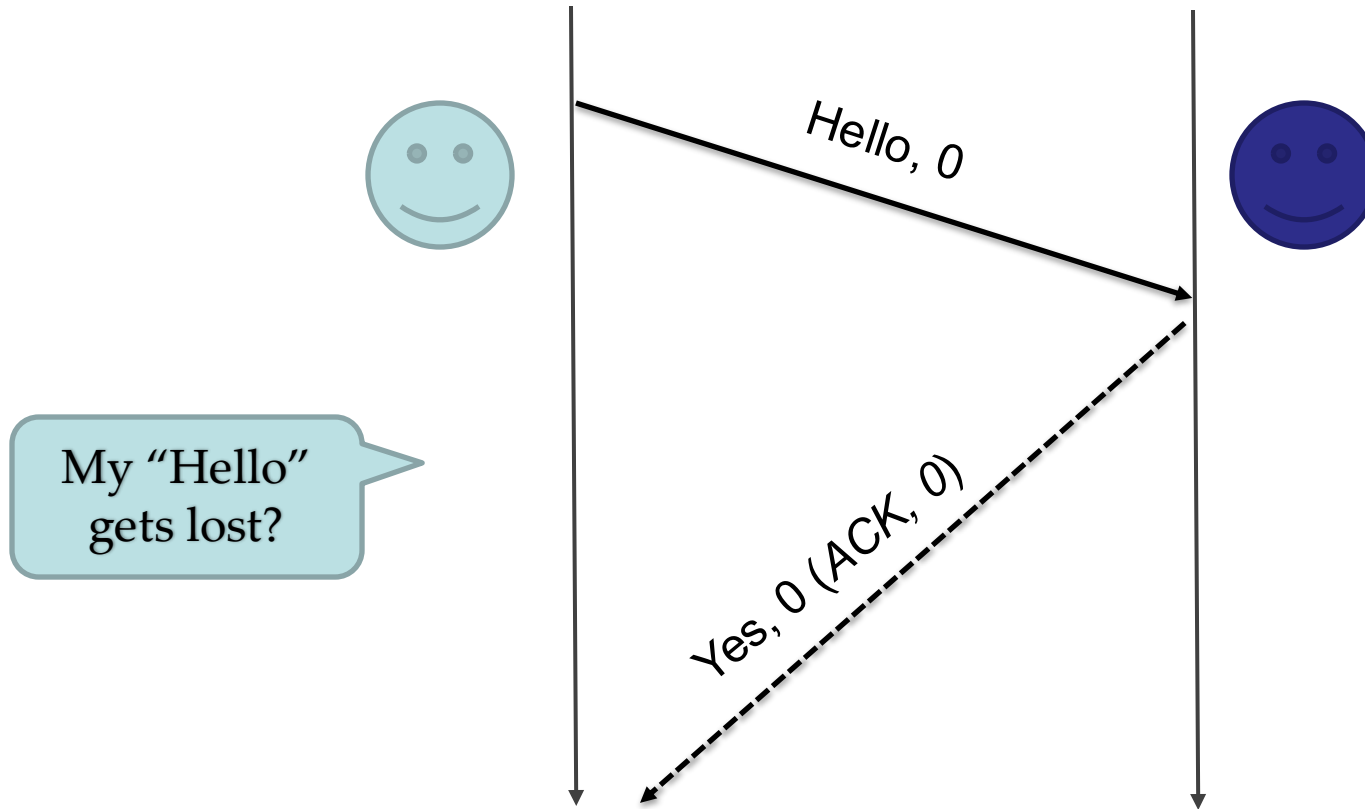
ACK Loss



Data Delayed



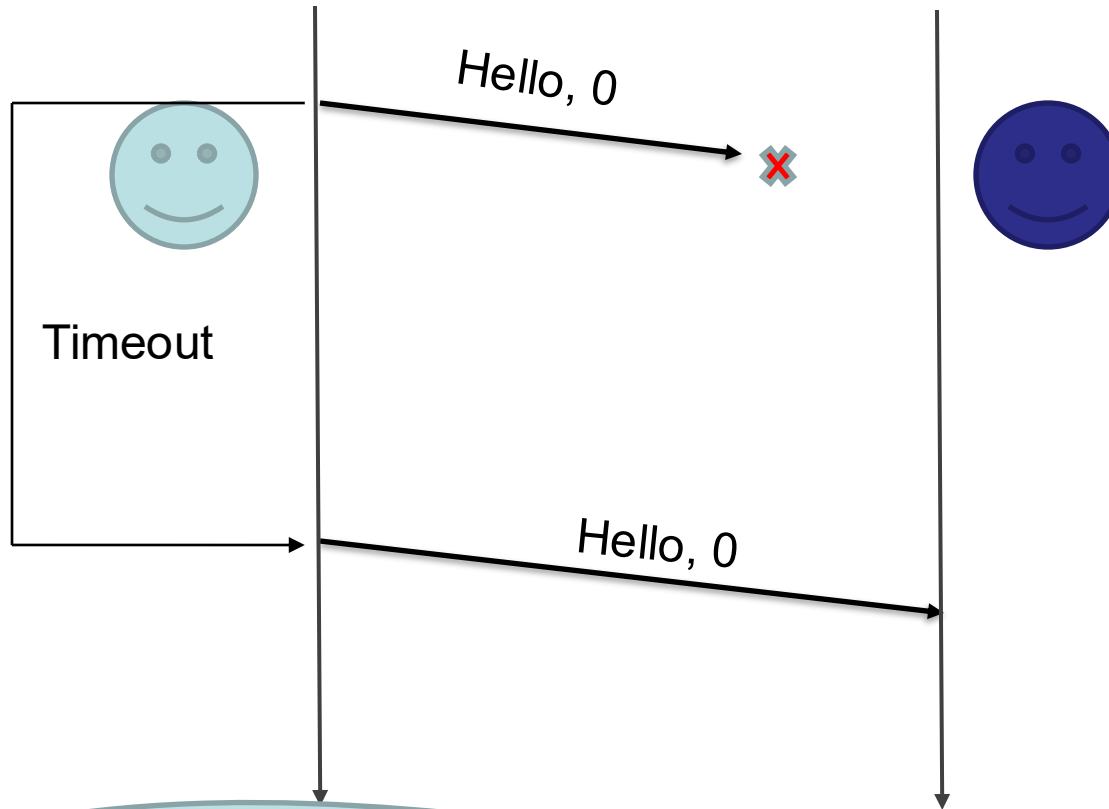
ACK Delayed



How to detect the packet loss?

- Wait, wait and wait!
- How long should the sender wait?
 - Too short → duplicate data packets
 - Too long → decreased performance
 - The solution: to judiciously choose a time value

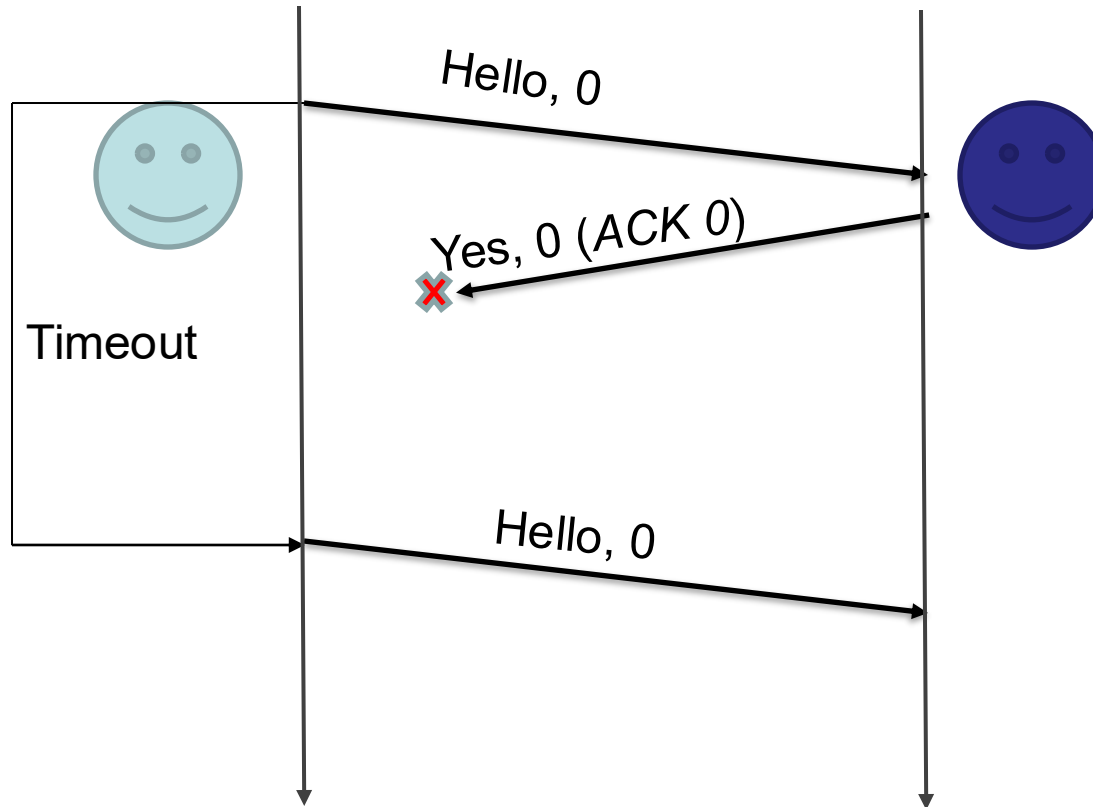
Example: Data Loss



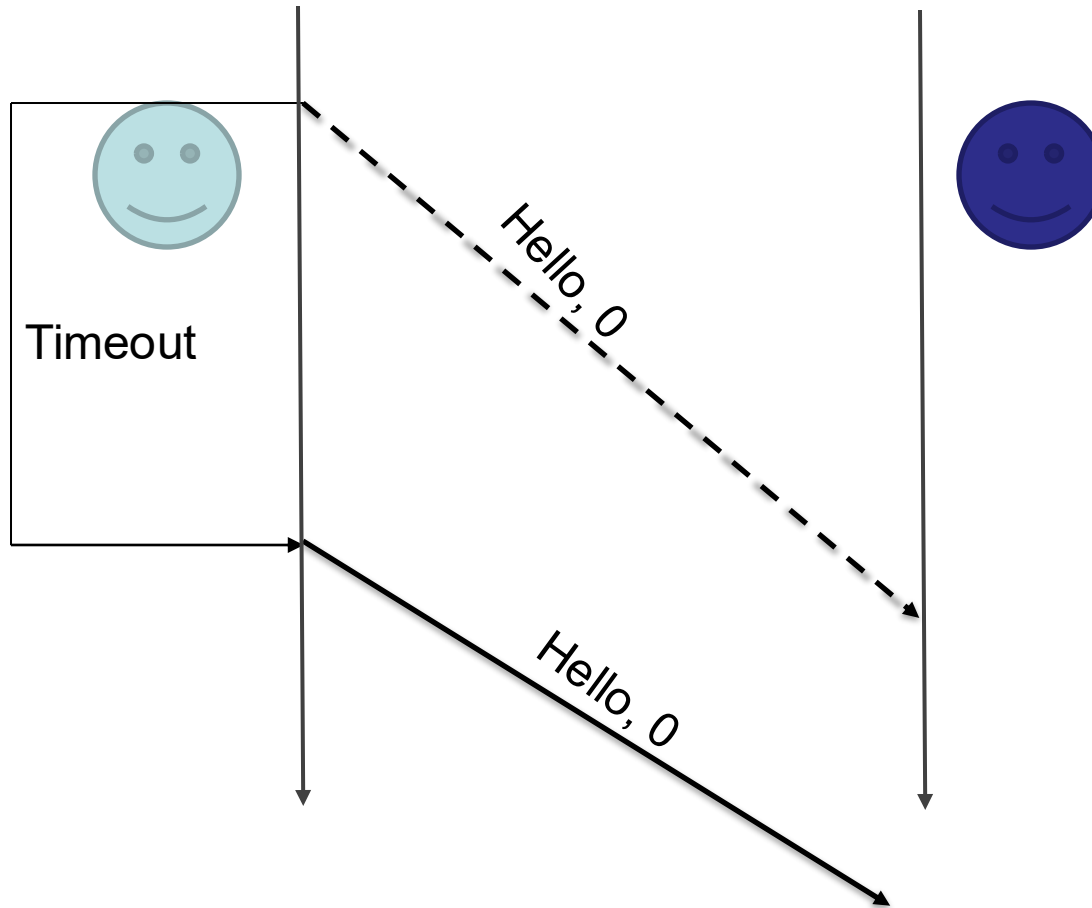
Use **timeout** to detect packet loss

Use **retransmission** to recover from packet loss

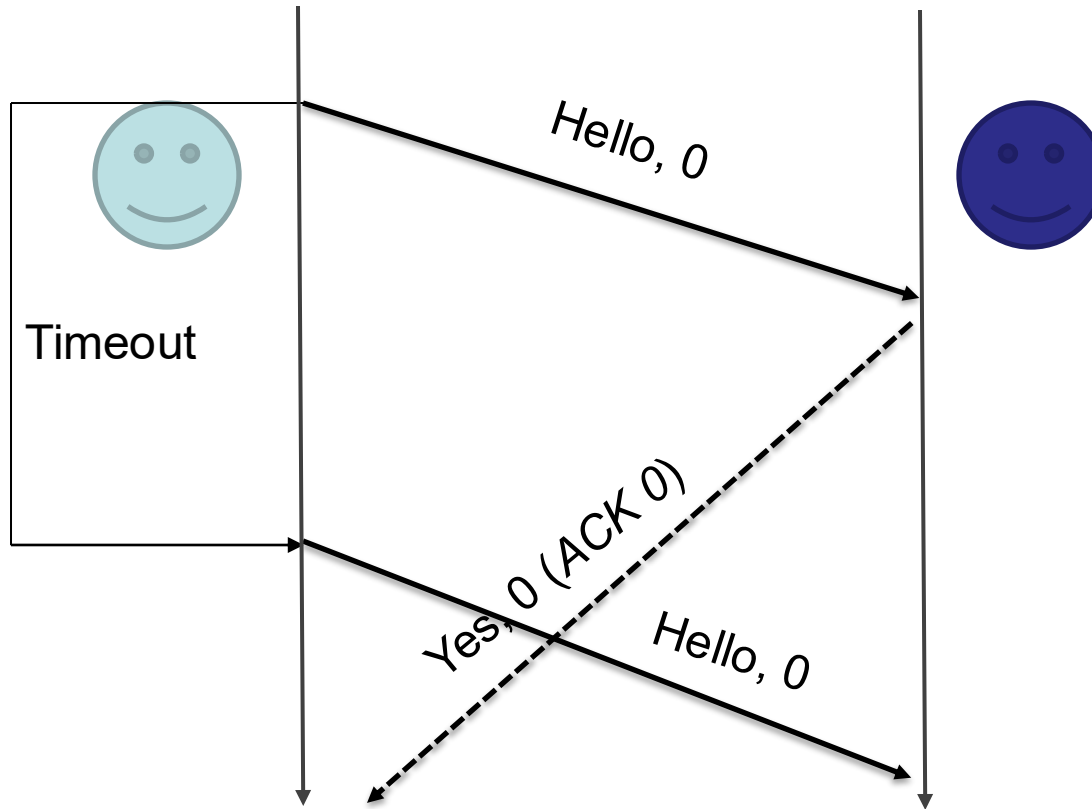
Example: ACK Loss



Example: Data Delayed



Example: ACK Delayed

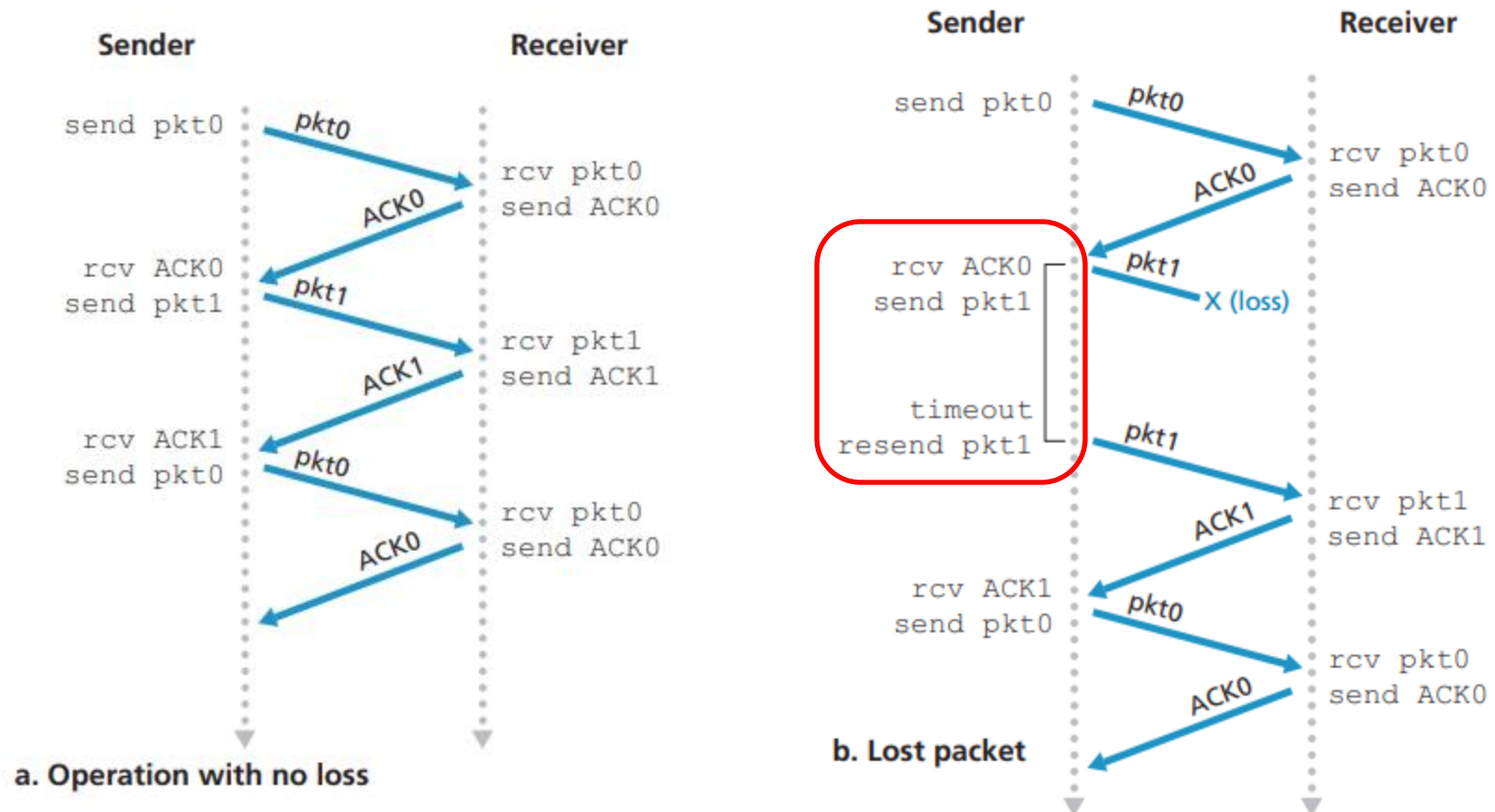


How to detect the packet loss? (cont.)

Use timeout to detect packet loss (*choose a time value wisely*)

Use retransmission to recover from packet loss
(*duplicate data packets – use seq#*)

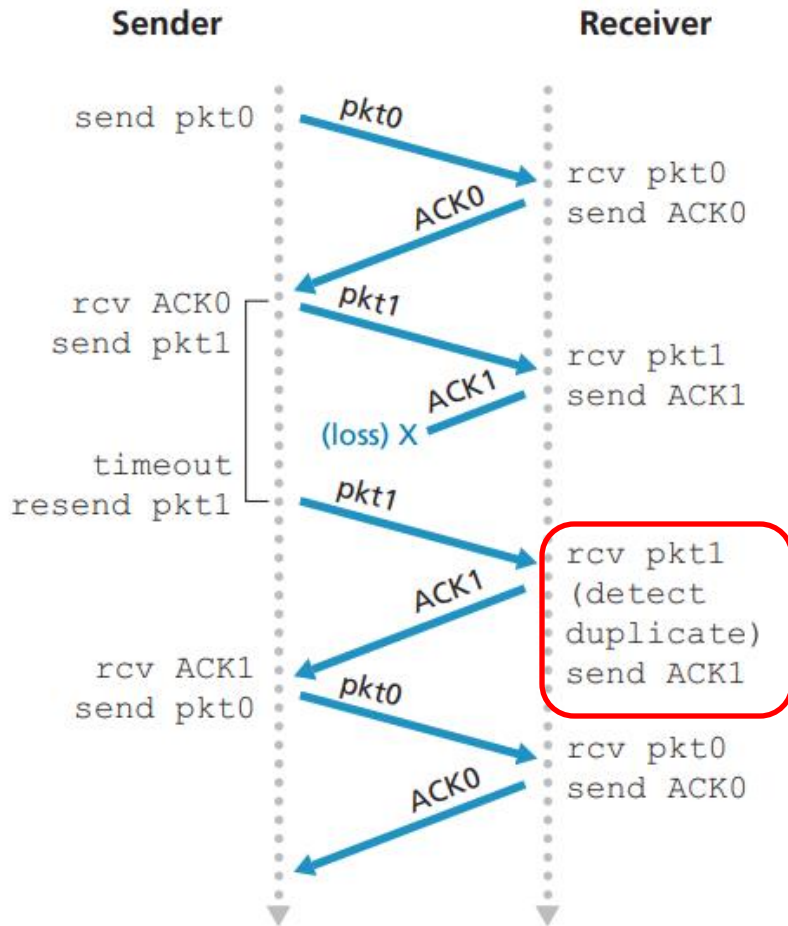
Alternating Bit Protocols: rdt 3.0



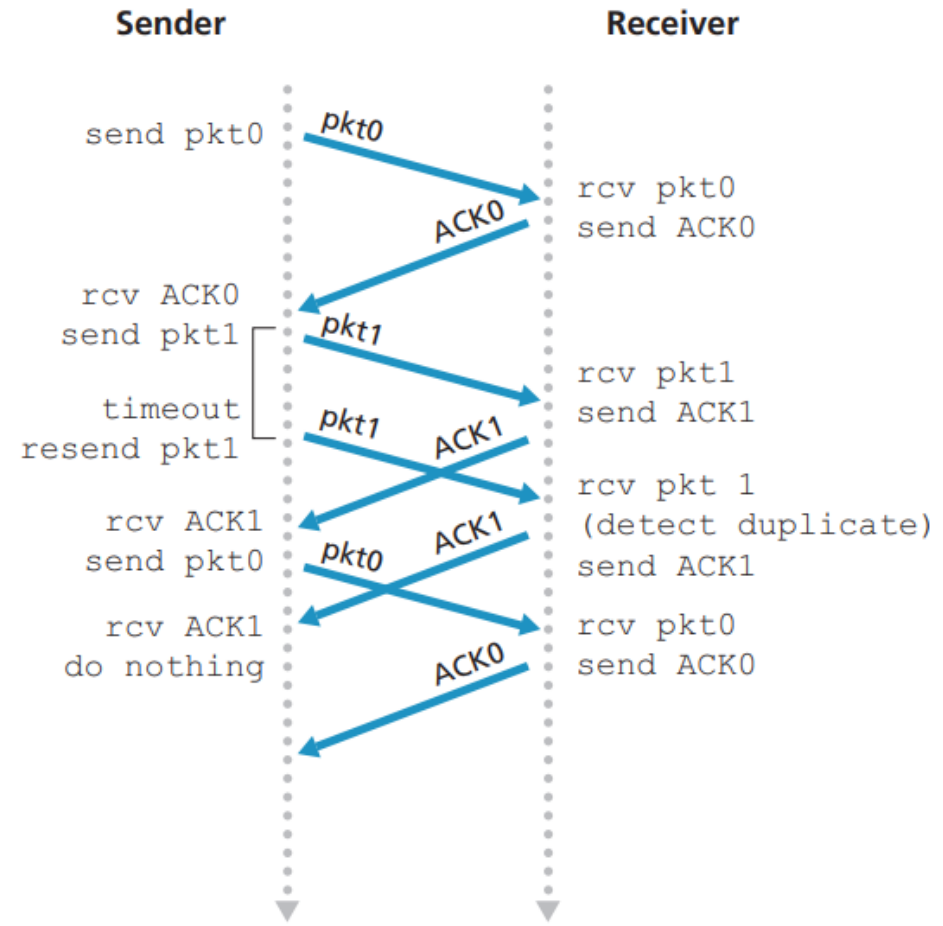
Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.16.

In-Confidence

Alternating Bit Protocols: rdt 3.0 (cont.)



c. Lost ACK



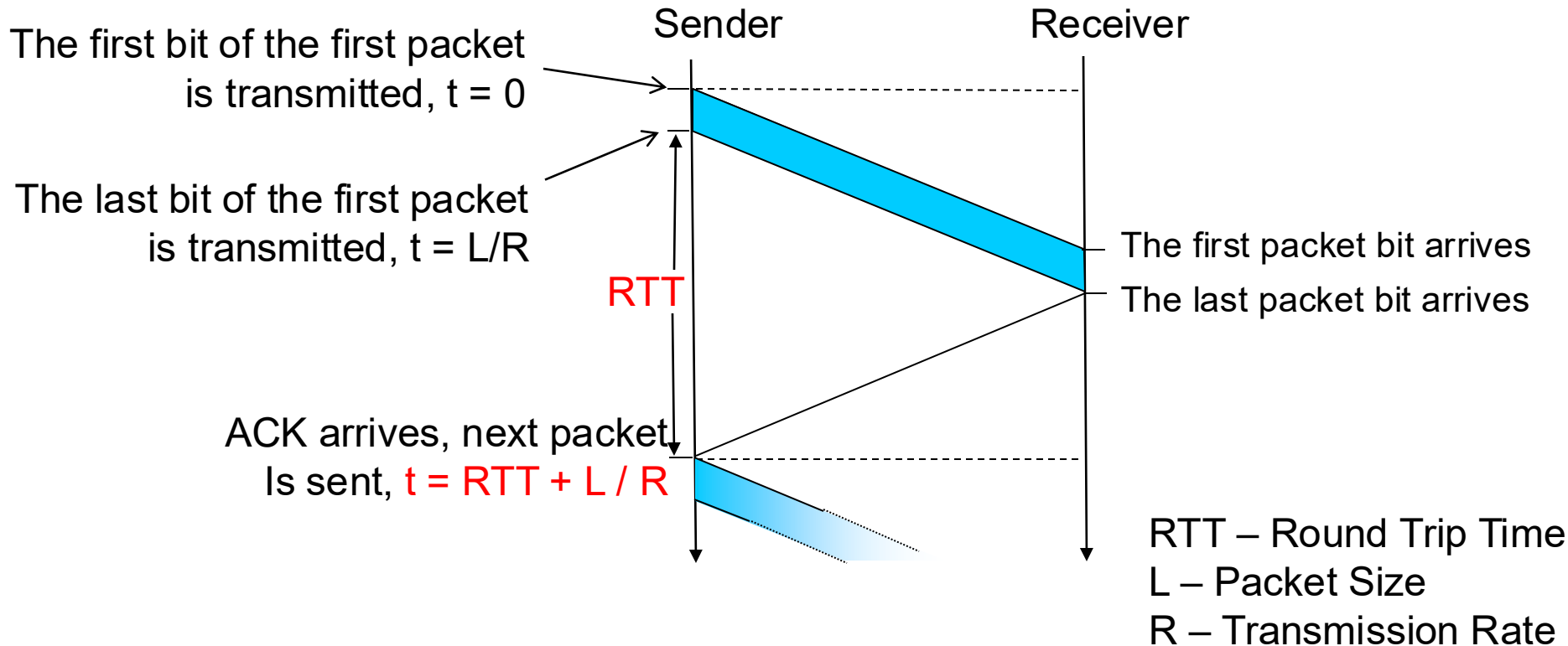
d. Premature timeout

Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.16.

In-Confidence

The Performance Problem of Alternating Bit Protocols

- It is essentially a stop-and-wait protocol.



$$\text{Utilisation of the sender: } U_{\text{sender}} = \frac{L/R}{RTT + L/R}$$

The Performance Problem of Alternating Bit Protocols (cont.)

- An example: transmission rate of 1 Gbps, RTT of 30 milliseconds , packet size of 1KB

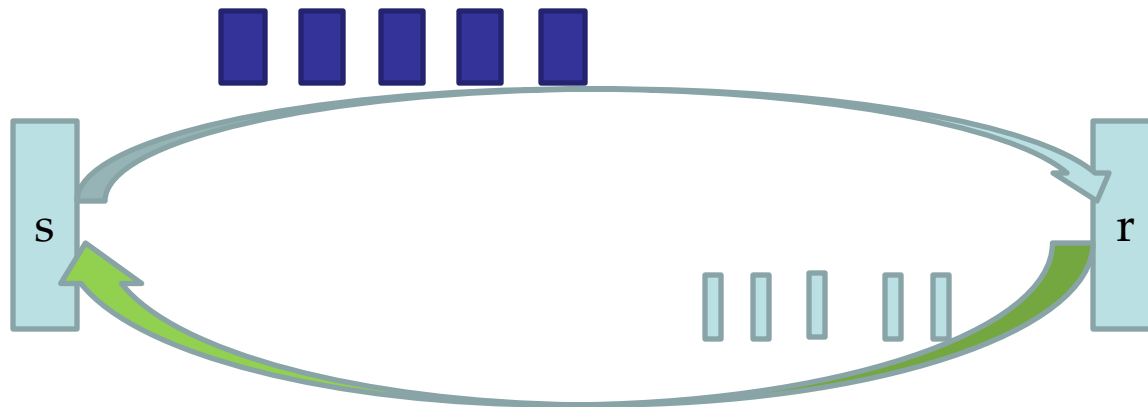
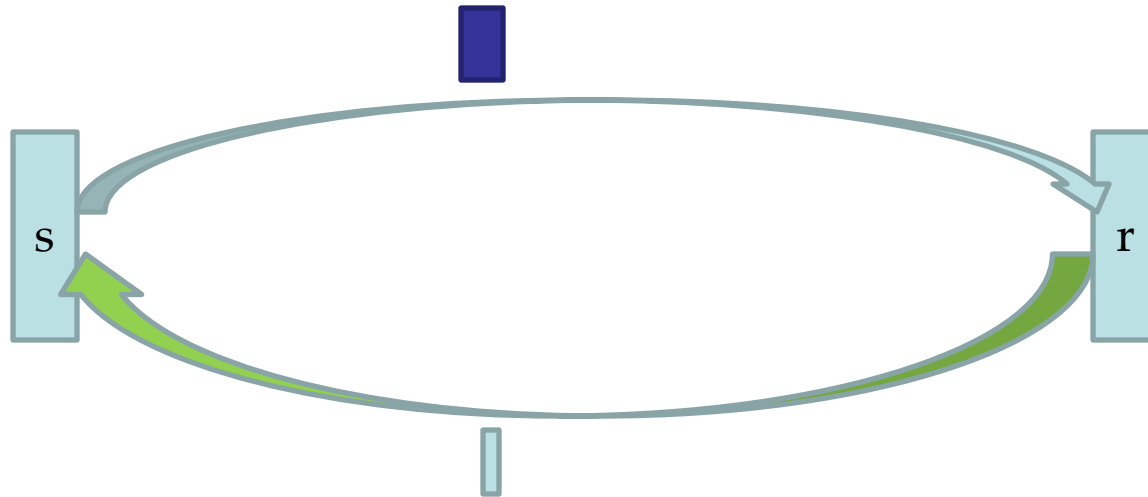
$$t_{trans} = \frac{L}{R} = \frac{8kb/pkt}{10^9 bits/sec} = 8 \text{ microseconds}$$

$$U_{sender} = \frac{L/R}{RTT + L/R} = \frac{.008}{30.008} = 0.00027$$

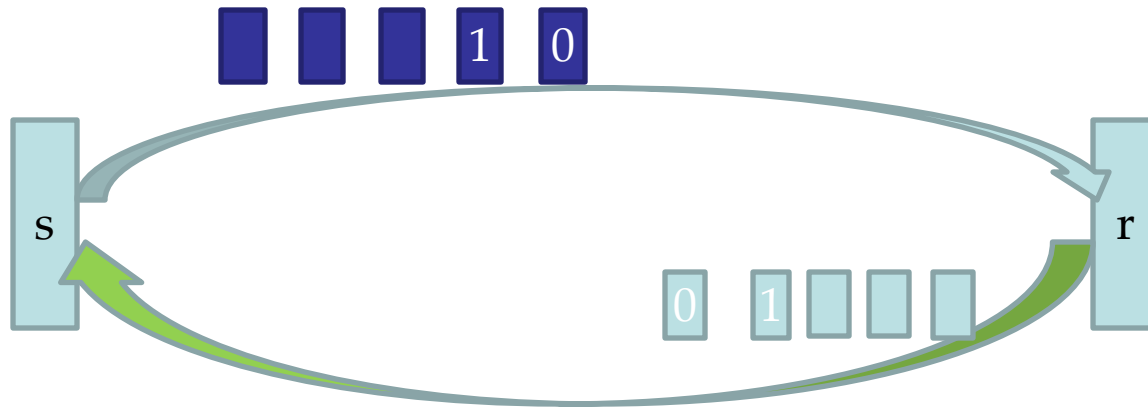
The sender is idle in most time – very low utilisation!
Not to mention we have neglected the lower layer protocol processing time at the sender and receiver, and processing and queuing delays at intermediate routers.

Pipelined reliable data transfer protocols

Pipelining



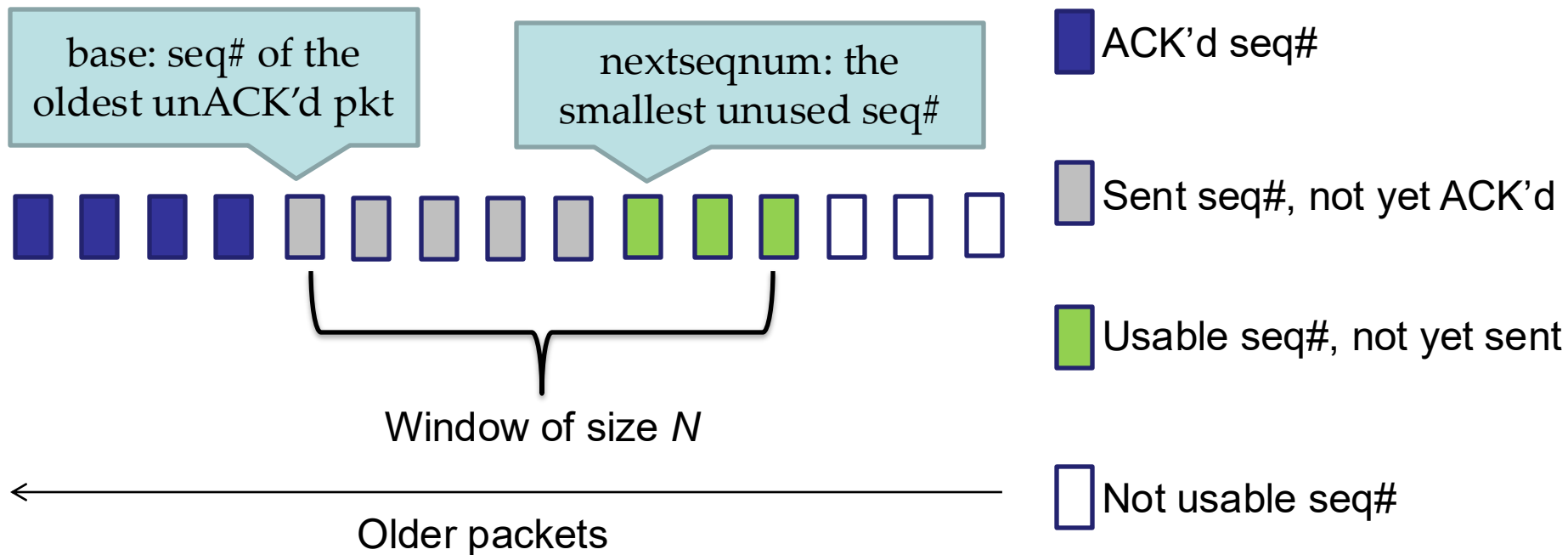
Pipelining (cont.)



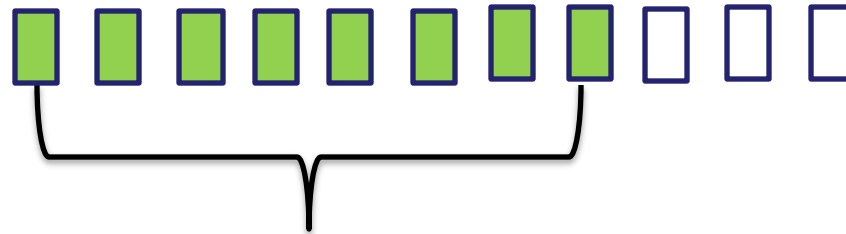
- Using a sequence number of $[0,1]$ is insufficient
 - In practice, a sequence number is k bits, ranging from 0 to $2^k - 1$ (TCP uses 32 bits)
- Sender and receiver may have to buffer more than one packet

Go-Back-N Protocol (Sliding-Window Protocol)

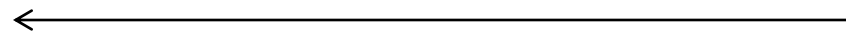
- The sender is allowed to send multiple packets without waiting for an acknowledgement but is constrained to have *no more than N unacknowledged packets* in the pipeline.



Go-Back-N: Invocation from Above (Sender)



Window of size N



Older packets

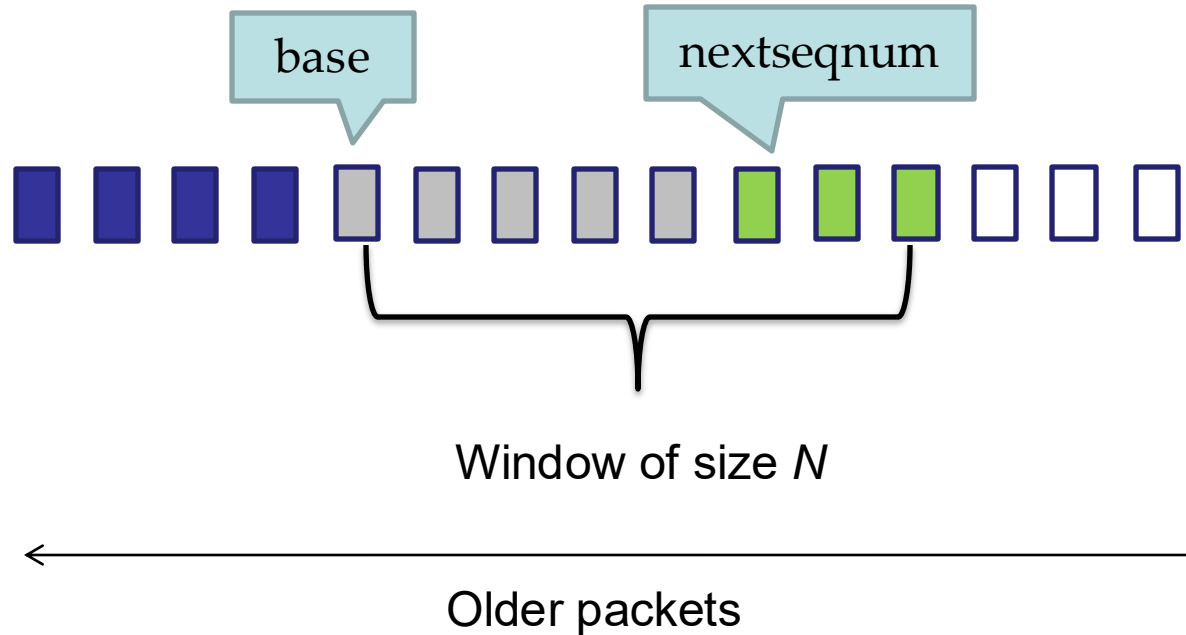
 ACK'd seq#

 Usable seq#, not yet sent

 Sent seq#, not yet ACK'd

 Not usable seq#

Go-Back-N: Receipt of an ACK (Sender)



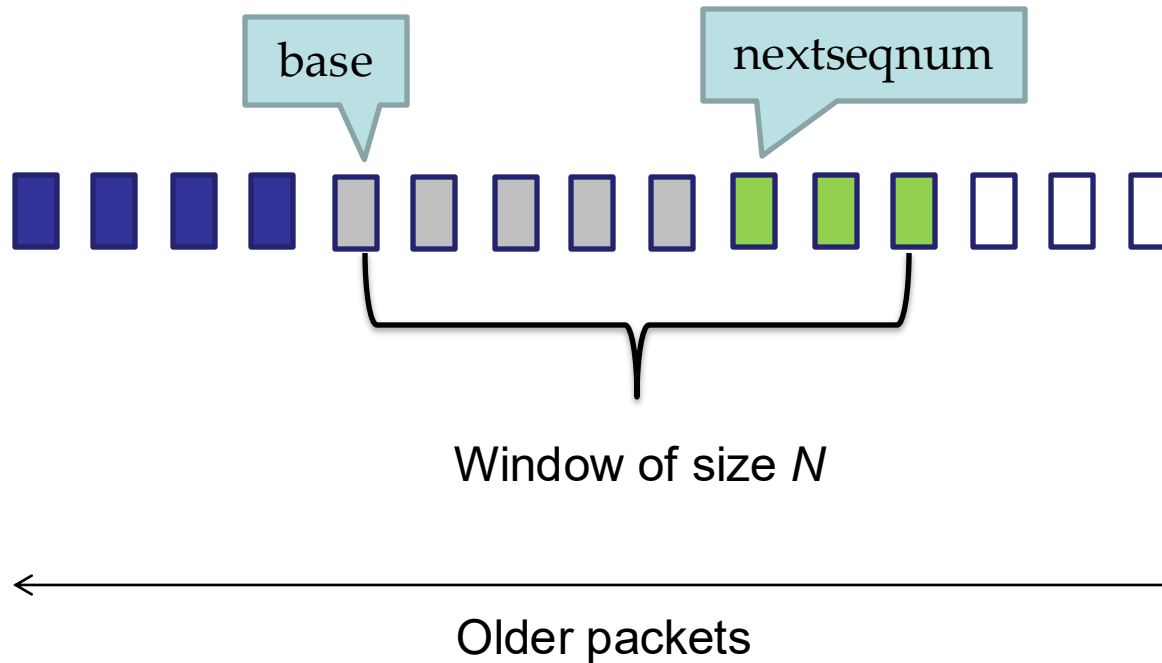
■ ACK'd seq#

■ Usable seq#, not yet sent

■ Sent seq#, not yet ACK'd

□ Not usable seq#

Go-Back-N: Cumulative ACK (Sender)



 ACK'd seq#

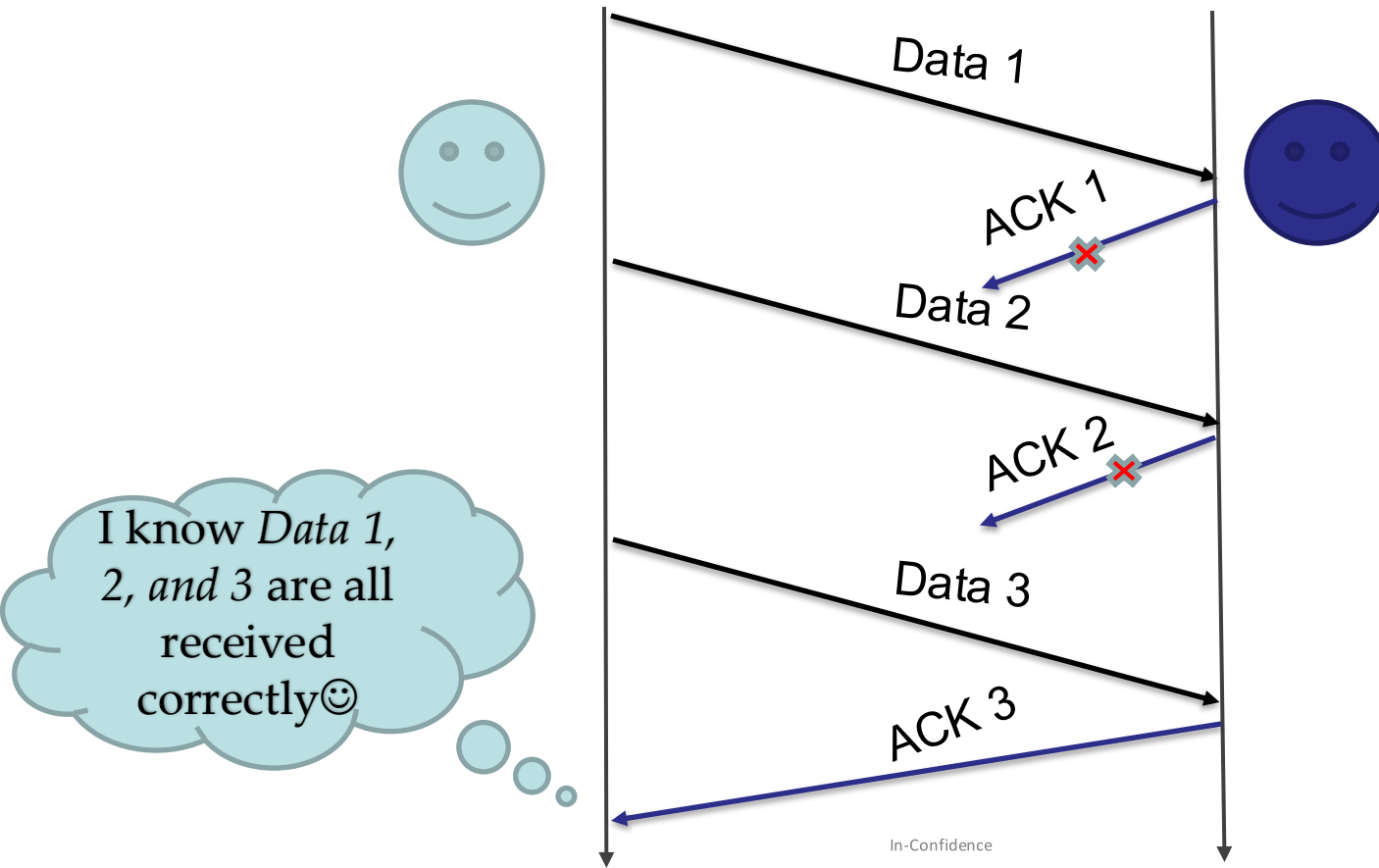
 Usable seq#, not yet sent

 Sent seq#, not yet ACK'd

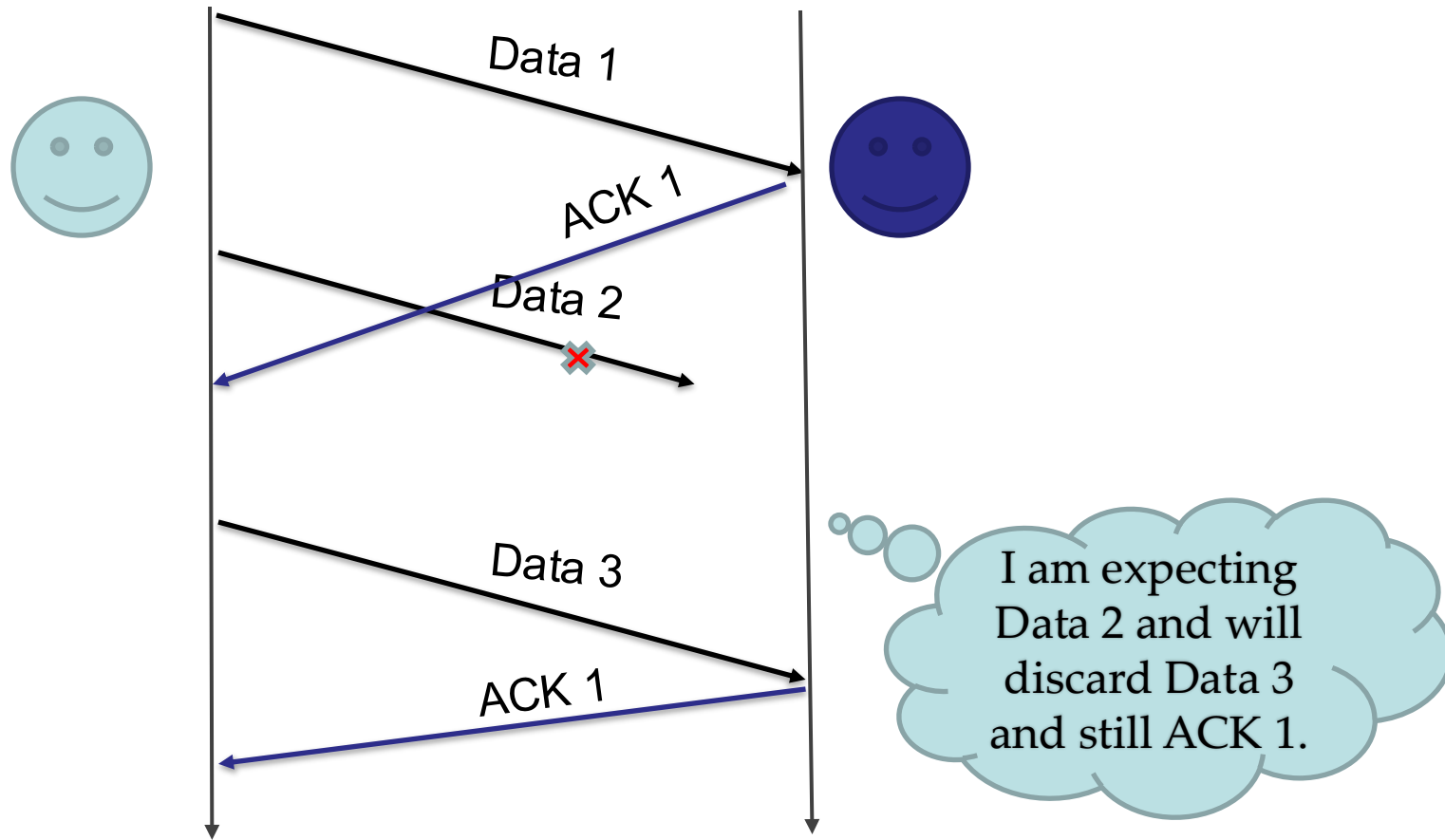
 Not usable seq#

Go-Back-N: Cumulative ACK (Sender) (cont.)

- An ACK with seq# n means that all packets with a seq# up to (and including) n have been correctly received at the receiver.

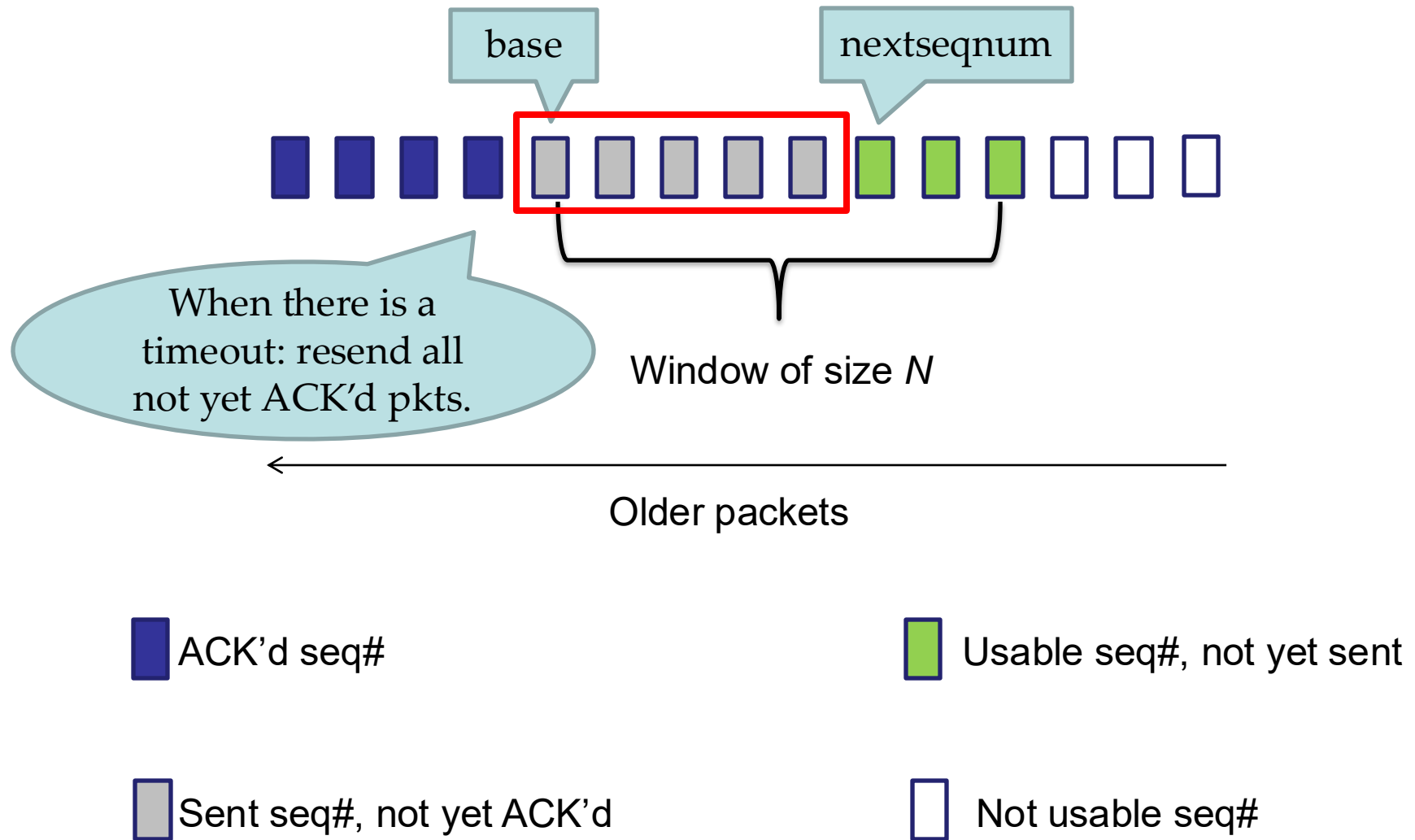


Go-Back-N: Cumulative ACK (Receiver)



Rule for Receiver: I'm always expecting the next in-order pkt; if unexpected pkt comes, I always discard them and resend an ACK for the latest correctly received in-order pkt.

Go-Back-N: Timeout (Sender)



Go-Back-N: Rules for the Receiver Side

Initially, expecting packet 0



When packet 0 arrives uncorrupted:

Deliver data to above;

Send ACK 0;

Now expecting packet 1.



Then packet 2 (*out-of-order*) arrives uncorrupted or a corrupted packet arrives:

Discard it;

Still send ACK 0;

Still expecting packet 1.



The receiver could have buffered pkt 2 for later delivery; but if pkt 1 is lost, pkts 1 & 2 will be retransmitted later.

For simplicity!

Then packet 1 arrives uncorrupted:

Deliver data to above;

Send ACK 1;

Now expecting packet 2.



The receiver only needs to keep track of the expected seqnum.

Go-Back-N: Extended FSM Descriptions

- Extended FSM of the sender
- Extended FSM of the receiver

Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.20 & 3.21.

Go-Back-N: Extended FSM of Sender

rdt_send(data)

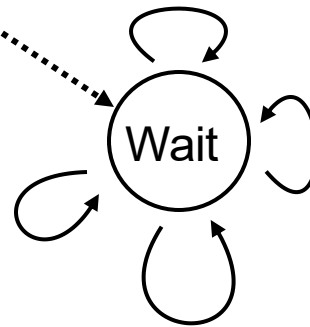
```

if (nextseqnum < base+N) { /*If we are allowed to send packets*/
    sndpkt[nextseqnum] = make_pkt(nextseqnum,data,chksum)
    udt_send(sndpkt[nextseqnum])
    if (base == nextseqnum) /*If there are no previous packets in flight*/
        start_timer /*Start timer for the just-sent packet*/
    nextseqnum++ /*Move to the next smallest unused sequence#/
}
else
    refuse_data(data)
    
```

Λ
base=1
nextseqnum=1

rdt_rcv(rcvpkt)
&& corrupt(rcvpkt)

Λ



timeout

```

(re)start_timer
/*Resend all previously sent unACK'ed packets*/
udt_send(sndpkt[base])
udt_send(sndpkt[base+1])
...
udt_send(sndpkt[nextseqnum-1])
    } Go back
    
```

rdt_rcv(rcvpkt) &&
notcorrupt(rcvpkt)

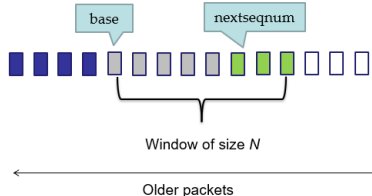
base = getacknum(rcvpkt)+1 /*Slide the window to its right*/

If (base == nextseqnum) /*If there are no packets in flight*/

stop_timer /* Stop timer because there are not outstanding packet*/

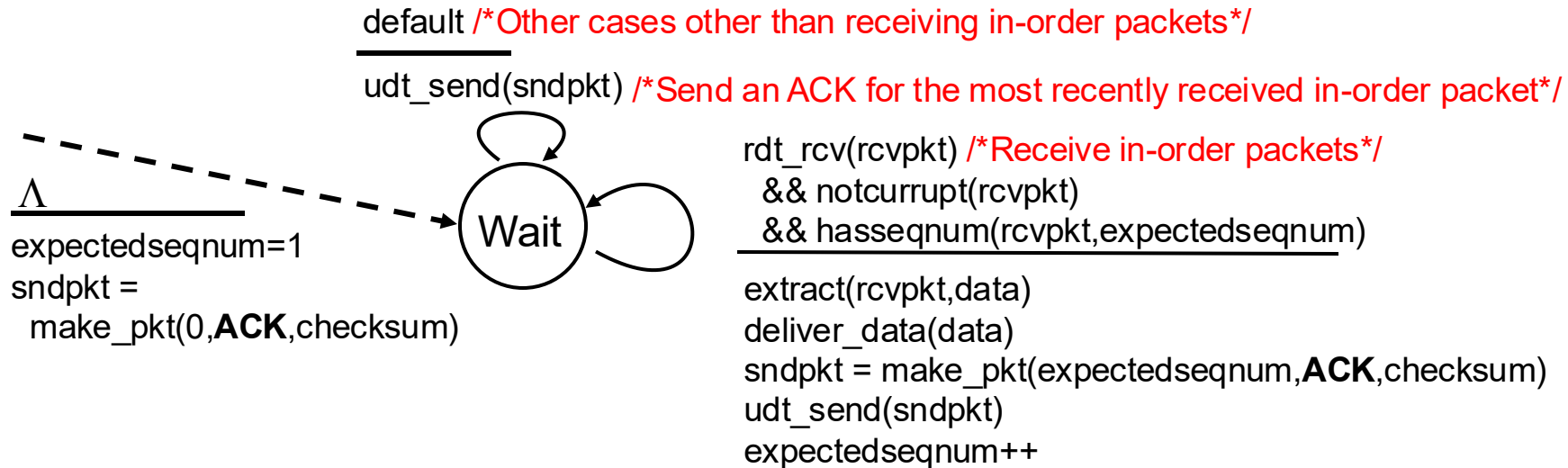
else

(re)start_timer /*Start timer for the oldest transmitted but not ACKed packet*/



- ACK'd seq#
- Usable seq#, not yet sent
- Sent seq#, not yet ACK'd
- Not usable seq#

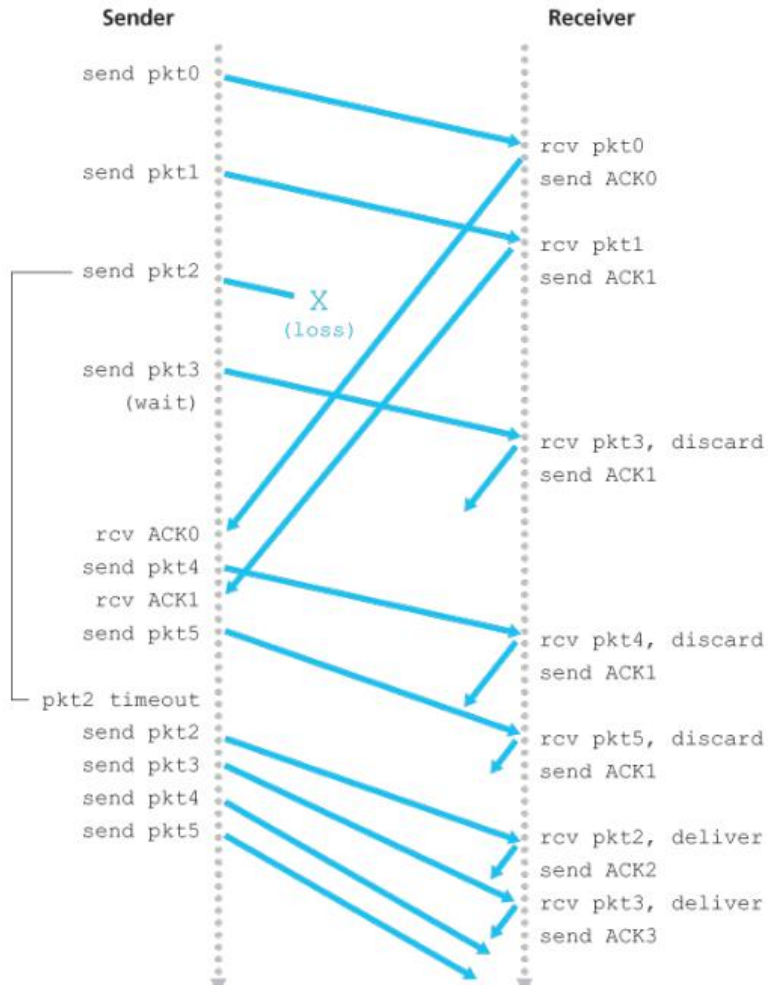
Go-Back-N: Extended FSM of Receiver



Receiver protocol:

If it receives an in-order packet; sends an ACK, delivers the packet, and expects the next in-order one; otherwise, it discards the received packet (out-of-order/corrupted); sends an ACK.

Go-Back-N Operation

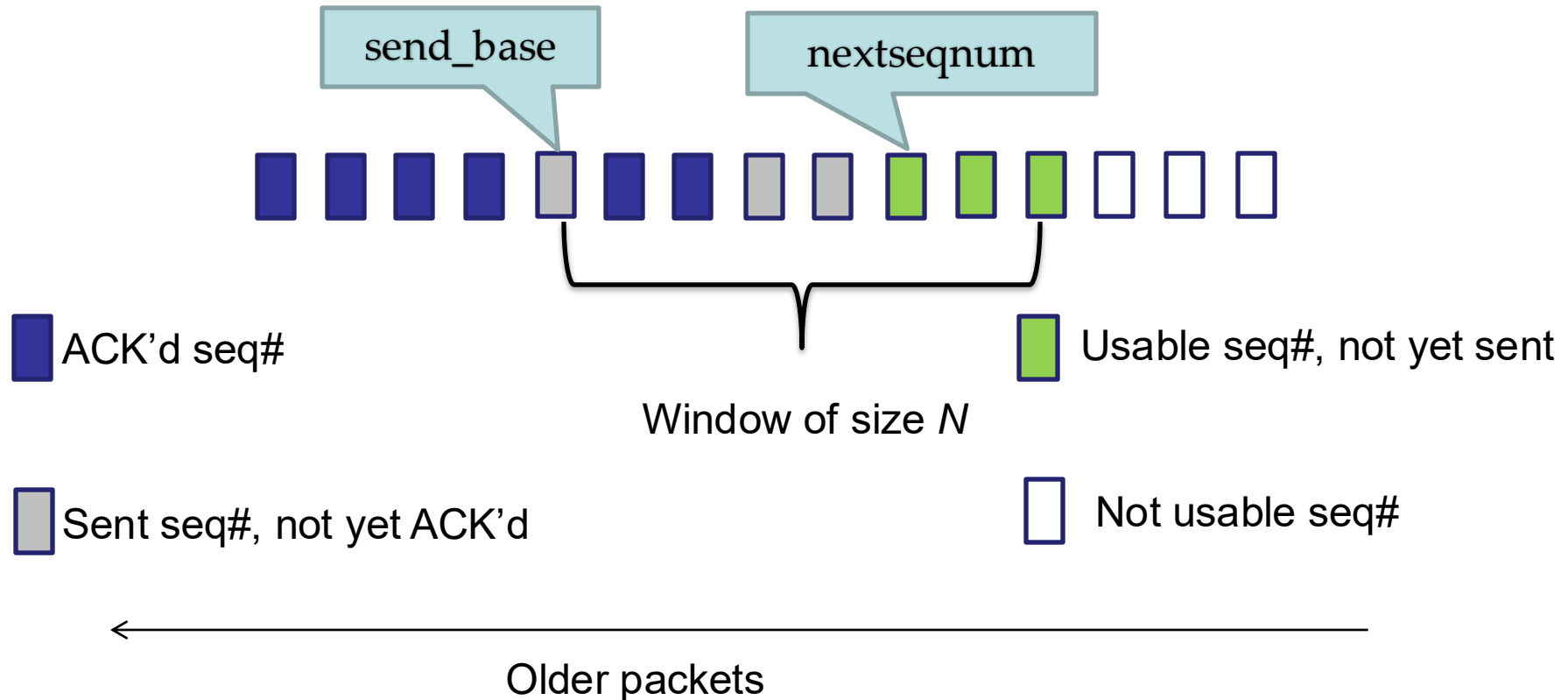


When there is a timeout caused by a single packet error, all unacknowledged packets will be *resent!*

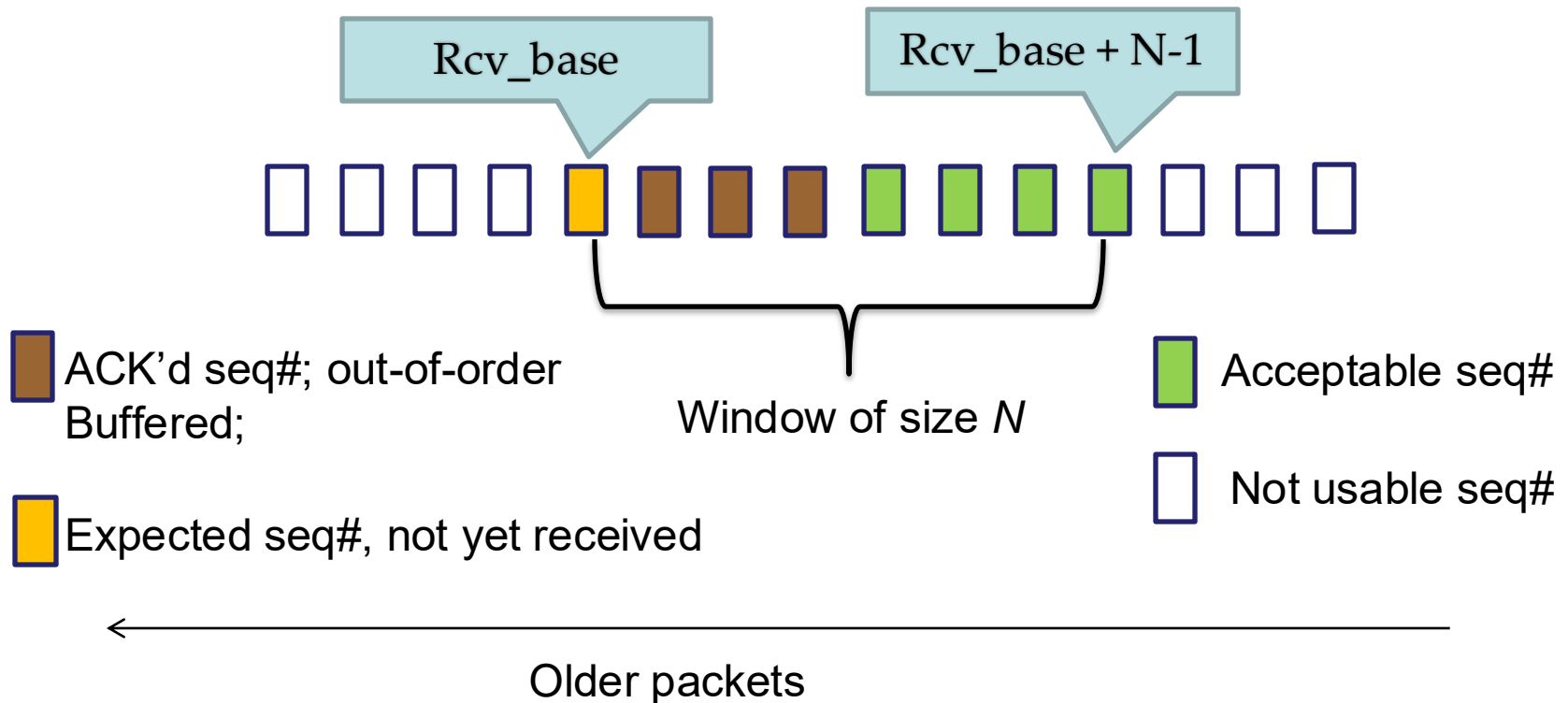
Selective Repeat/Reject (SR)

- receiver *individually* acknowledges all correctly received pkts
 - buffers pkts, as needed, for eventual in-order delivery to upper layer
- sender only resends pkts for which ACK not received
 - sender has timer for each unACKed pkt
- sender window
 - N consecutive seq #'s
 - again limits seq #'s of sent, unACKed pkts
- Receiver window
 - N consecutive seq #'s
 - defines the range of sequence numbers that the receiver is currently willing to accept and buffer.

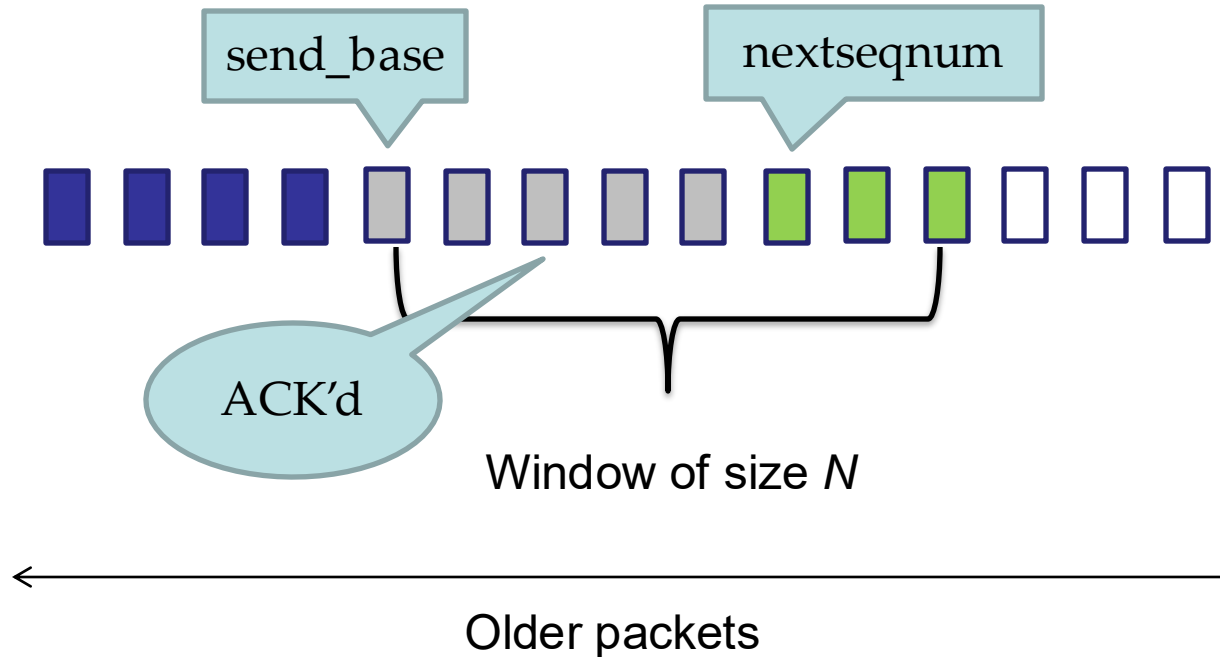
SR: Sender window



SR: Receiver window



SR: Sender Window – Receipt of an ACK



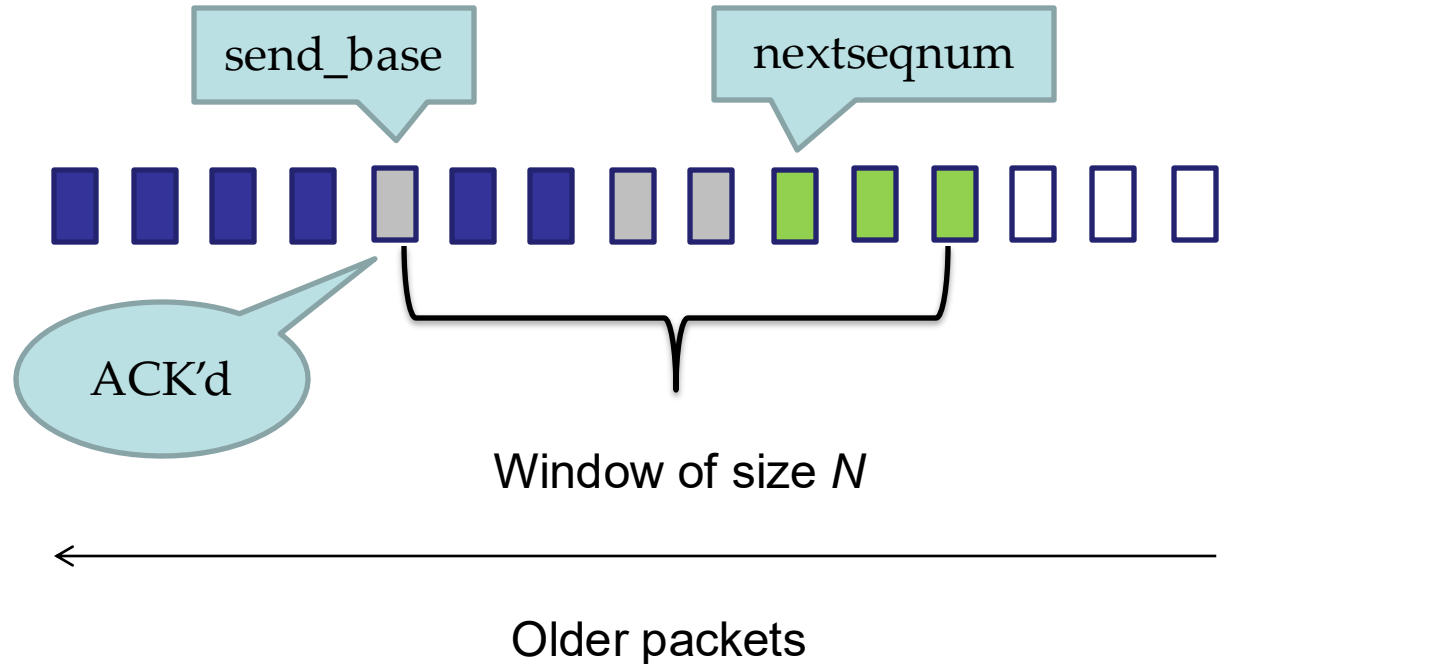
■ ACK'd seq#

■ Usable seq#, not yet sent

■ Sent seq#, not yet ACK'd

□ Not usable seq#

SR: Sender Window – Receipt of an ACK (cont.)



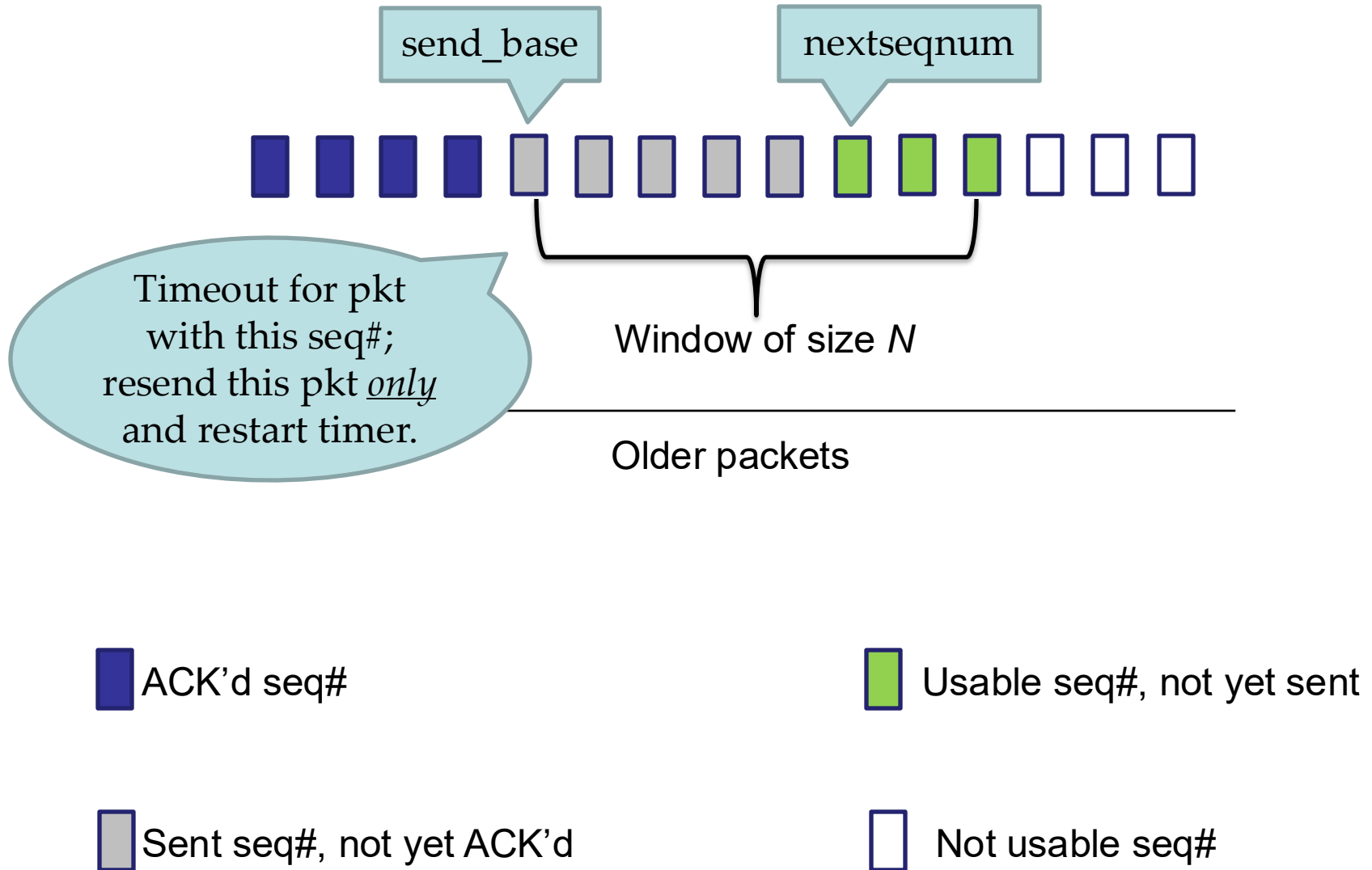
■ ACK'd seq#

■ Usable seq#, not yet sent

■ Sent seq#, not yet ACK'd

□ Not usable seq#

SR: Sender Window – Timeout



SR: Sender Side

Sender

Data received from above

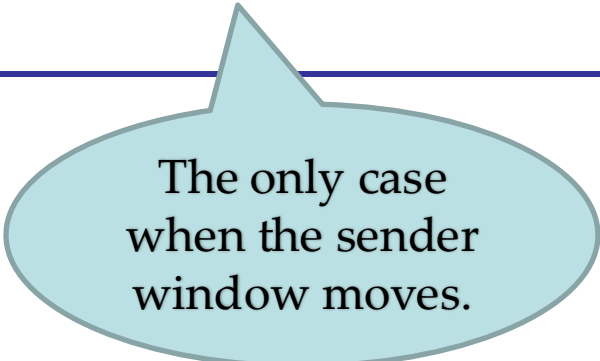
- if the next available seq# is in the window, send the packet

Timeout

- each packet has its own timer
- resend the packet and restart the timer

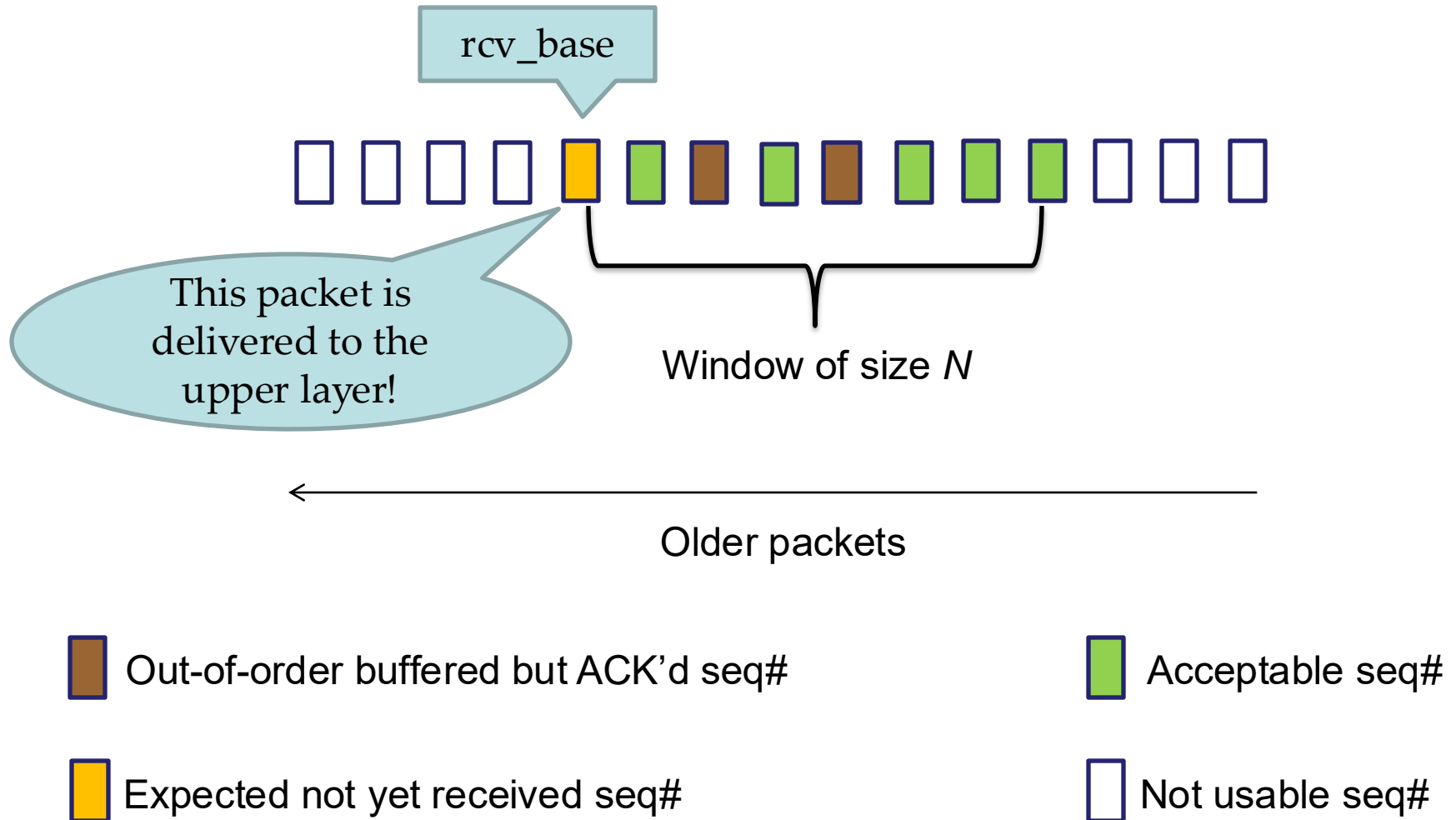
ACK received with seq# in [send_base, send_base+N-1]

- mark the packet as received
- if the seq# = send_base, advance the window base to next unACK'd seq#

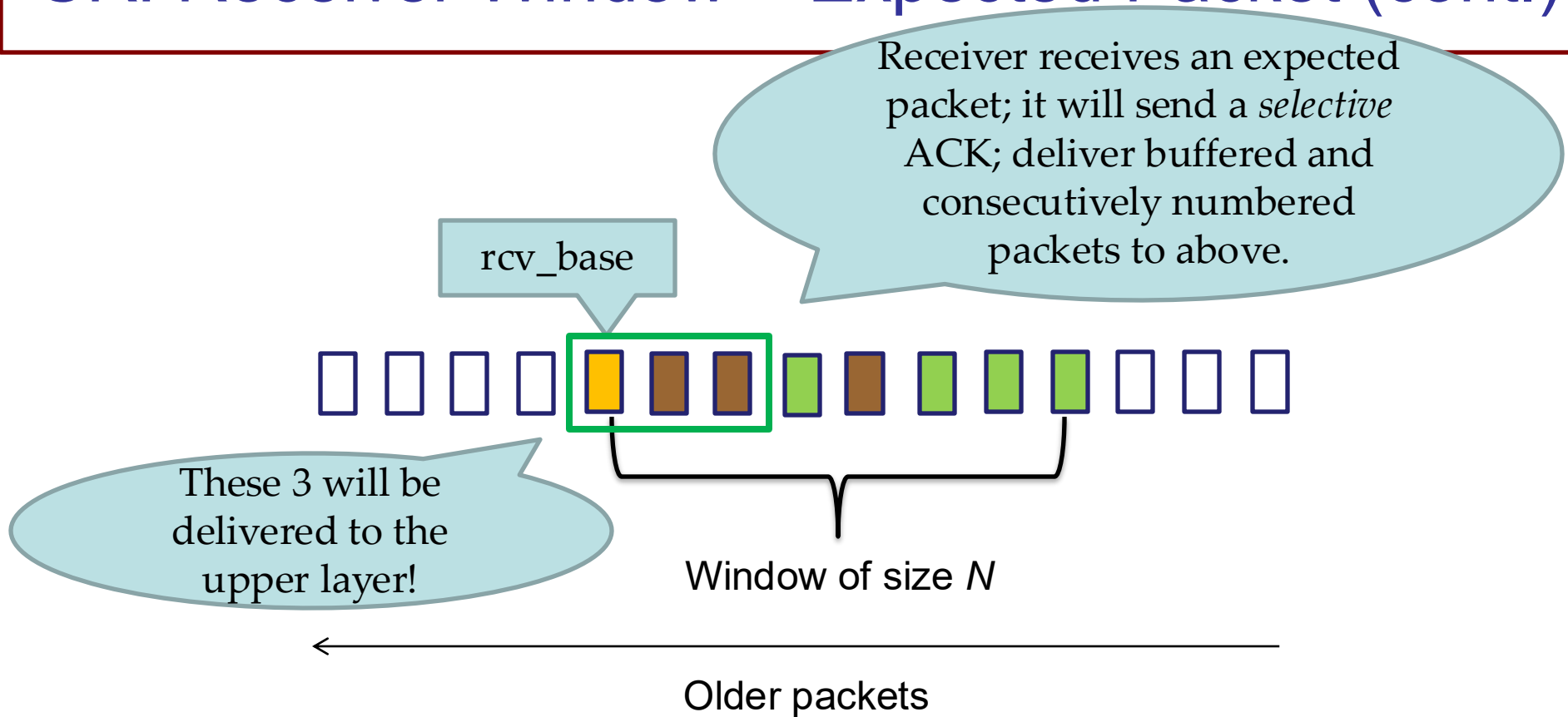


The only case
when the sender
window moves.

SR: Receiver Window – Expected Packet



SR: Receiver Window – Expected Packet (cont.)



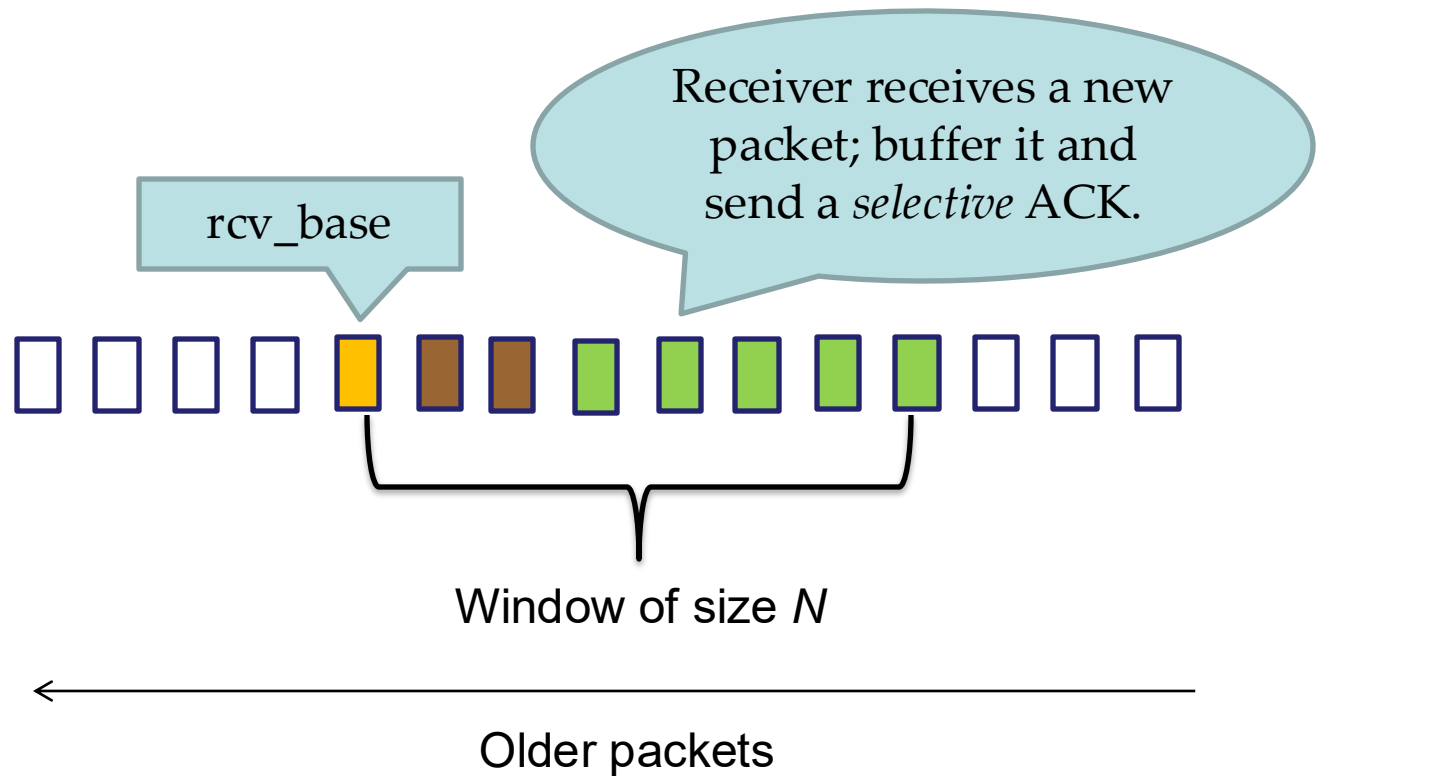
 Out-of-order buffered but ACK'd seq#

 Acceptable seq#

 Expected not yet received seq#

 Not usable seq#

SR: Receiver Window – New Out-of-Order Packet



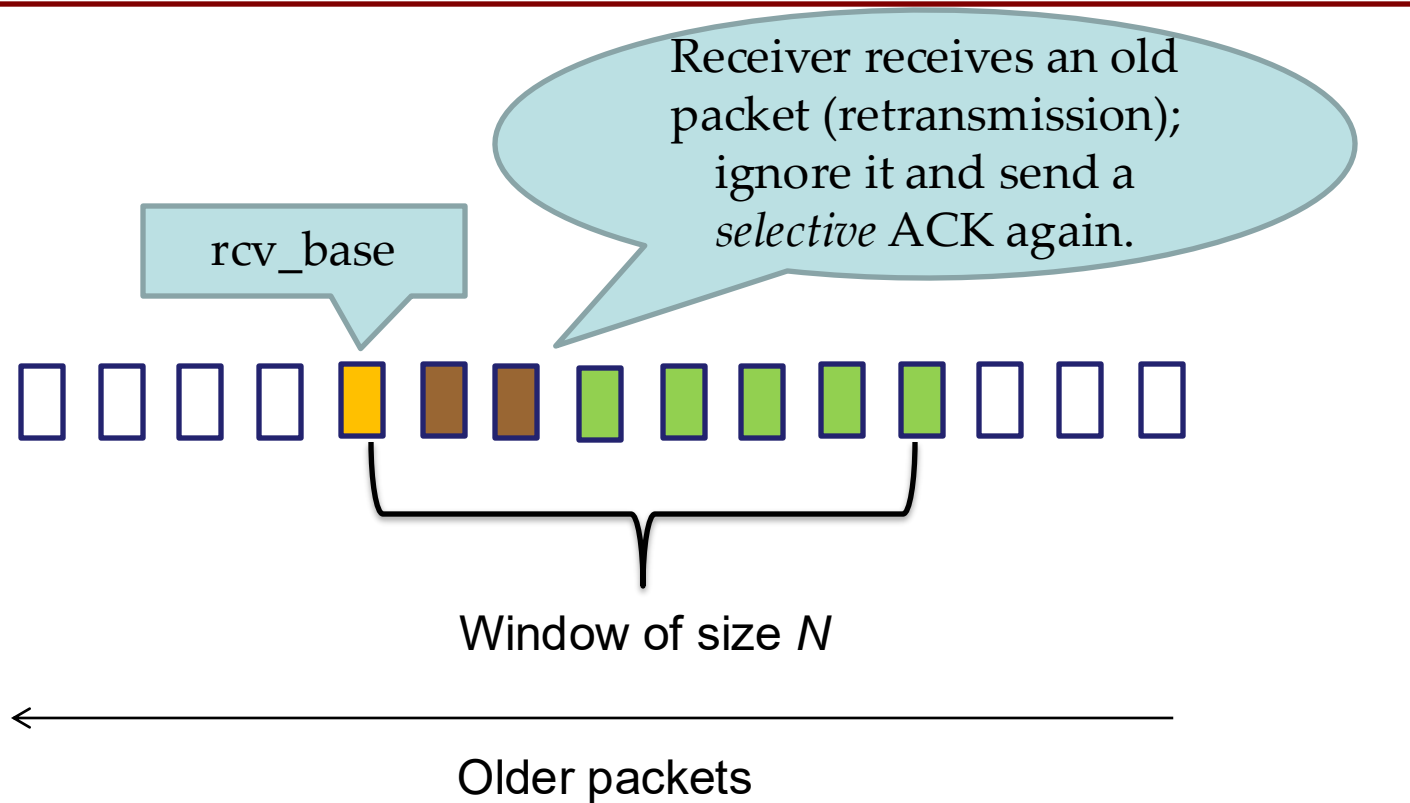
 Out-of-order buffered but ACK'd seq#

 Acceptable seq#

 Expected not yet received seq#

 Not usable seq#

SR: Receiver Window – Old Packet



 Out-of-order buffered but ACK'd seq#

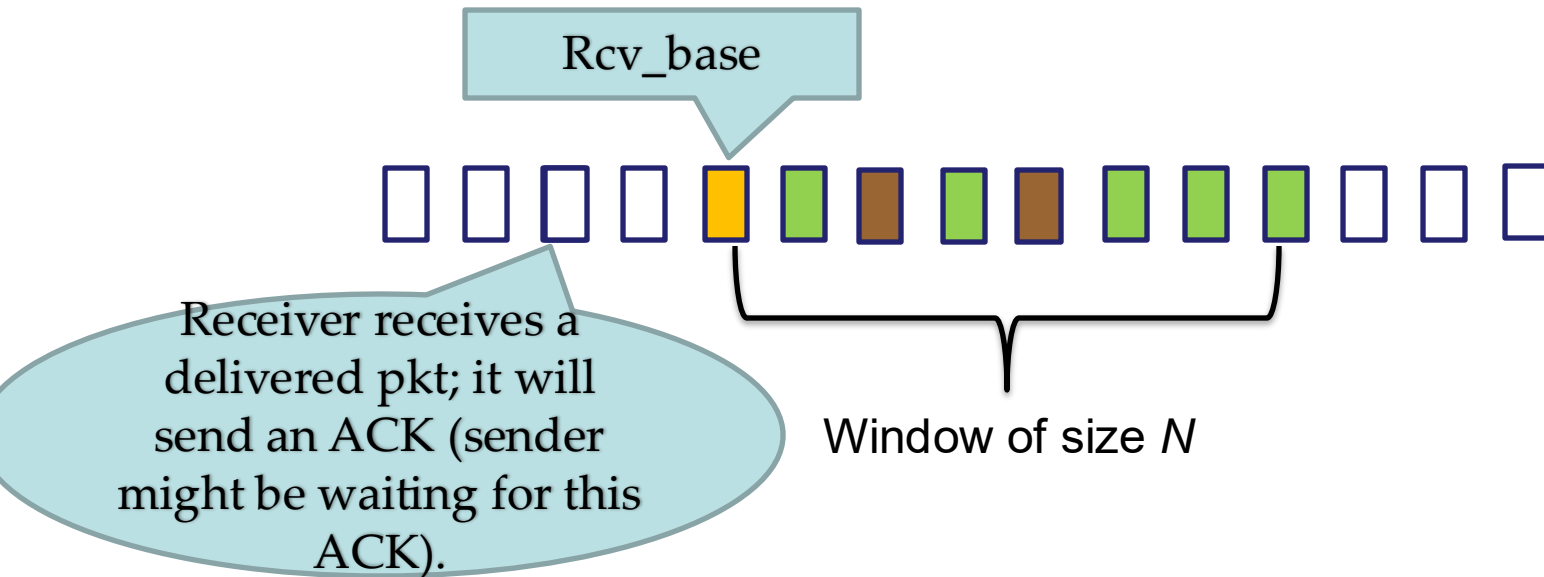
 Acceptable seq#

 Expected not yet received seq#

 Not usable seq#

SR: Receiver window – a delivered pkt - seq# in [rcv_base-N, rcvbase-1]

S: 0 1 2 3 4 5 6 7 8 9 R: 0 1 2 3 4 5 6 7 8 9



-  ACK'd seq#; out-of-order Buffered;
-  expected seq#, not yet received

-  acceptable seq#
-  Not usable seq#

SR: Receiver Side

Receiver

Packet with seq# in $[rcv_base, rcv_base+N-1]$

- ❑ send a selective ACK
- ❑ out-of-order: the packet is buffered
- ❑ if seq# = rcv_base, the packet and any buffered, in-order packets are delivered, advance the window by the number of packets delivered

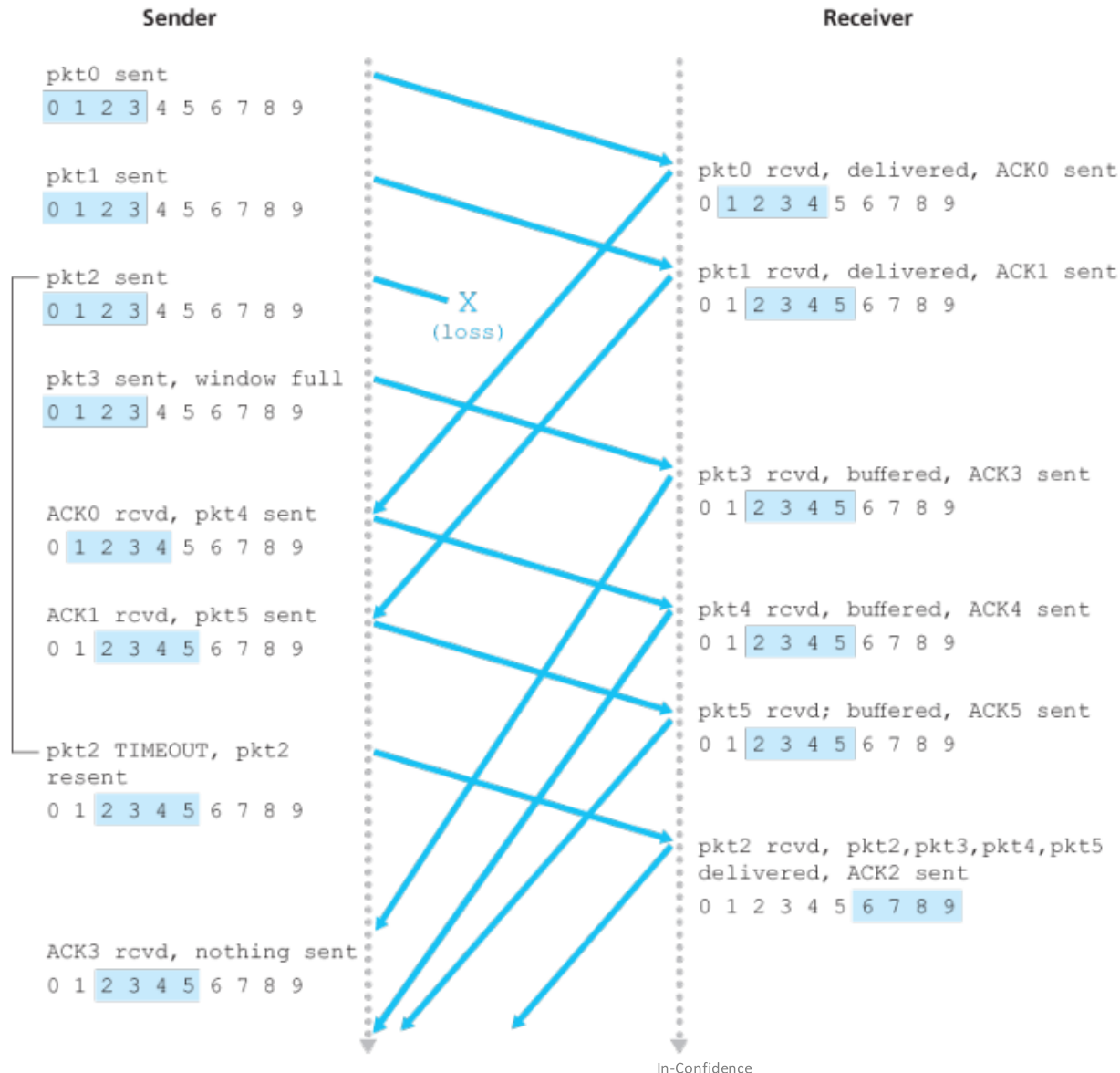
Packet with seq# in $[rcv_base-N, rcv_base-1]$

- ❑ send ACK even if the receiver has previously acknowledged

Otherwise

- ❑ ignore the packet

SR Operation



Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.26.

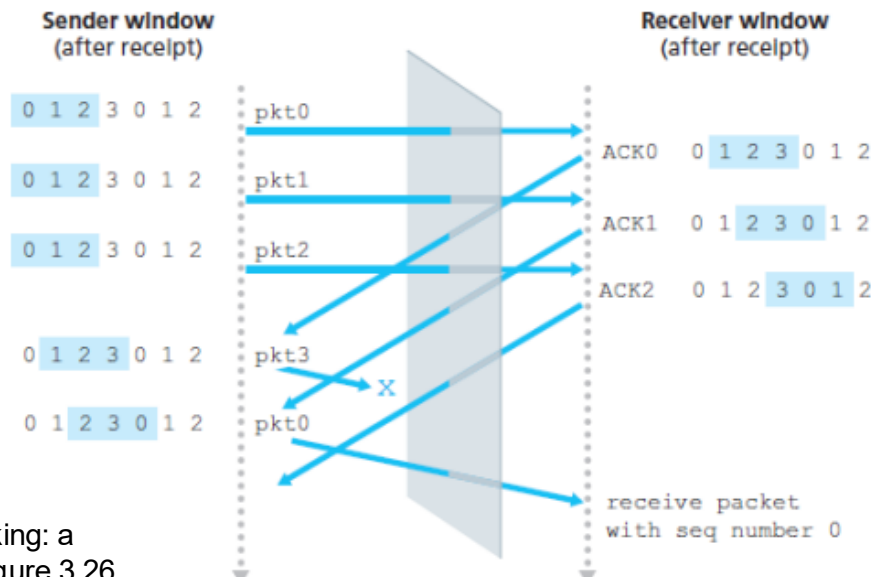
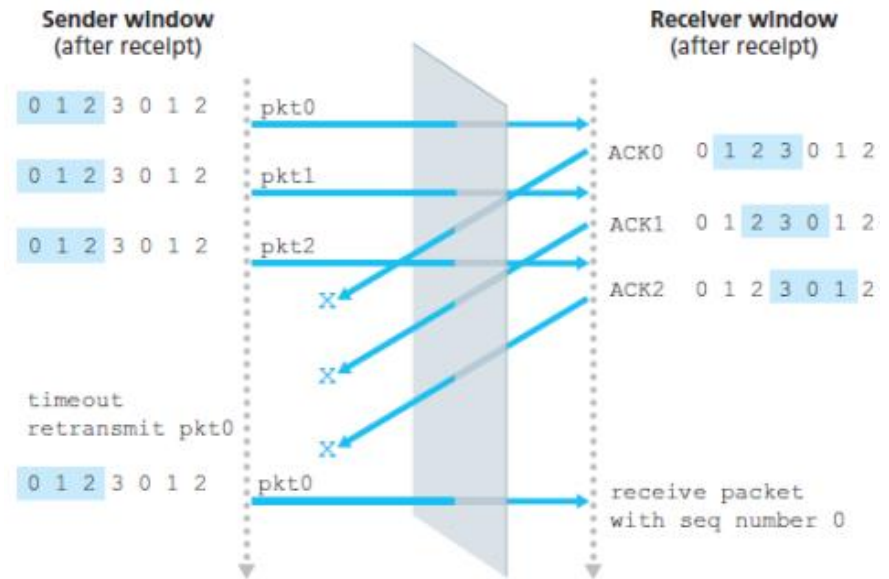
SR: Receiver Dilemma with Too Large Windows

Example:

- seq#: 0, 1, 2, 3
- window size = 3

A retransmission
with seq# 0

A new transmission
with seq# 0



Source: James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, 3rd edition, 2005, Figure 3.26.

What is the relationship between the window size and seq# space?

$$\text{window size} \leq \frac{1}{2} (\text{size of seq\# space})$$

Summary

Mechanism	Use, Comments
Checksum	To detect bit errors
Timer	To timeout/retransmit a packet (lost/premature packet)
Sequence number	To detect a lost packet (gap in seq#) To detect a duplicate packet (duplicate seq#)
Acknowledgement	To notify the successful receipt of packets (individual/cumulative)
Negative ACK	To notify the unsuccessful receipt of packets
Window, pipelining	To improve sender utilisation

References

- James F. Kurose & Keith W. Ross, Computer networking: a top-down approach featuring the Internet, Addison Wesley, 3rd edition, 2005.
 - Chapter 3 Transport Layer (3.4)
- William Stallings, Data and Computer Communications, Pearson, 10th edition, 2013.
 - Chapter 7 Data Link Control Protocols
- Andrew S. Tanenbaum & David J. Wetherall, Computer Networks, Prentice Hall, 5th edition, 2010.
 - Chapter 3 The Data Link Layer (3.3, 3.4)

Acknowledgements

- Lecture notes are developed based on slides from the following three sources:
 - Dr Mengmeng Ge's lecture notes for COSC264, University of Canterbury, 2024
 - Assoc Prof Dan Kim's lecture notes for COSC264, University of Canterbury, 2017
 - Prof Aleksandar Kuzmanovic's lecture notes for CS340, Northwestern University, 2005, https://users.cs.northwestern.edu/~akuzma/classes/CS340-w05/lecture_notes.htm